Decision-Aware Bellman Sampling for Static AVaR Control Using Gaussian Processes

Dilzhan Zhangunissov[1], Aruzhan Dauleshova[2], and Kerem Uğurlu[1]

[1]Department of Mathematics, Nazarbayev University, Kazakhstan
[2]Independent Researcher

November 3, 2026

Abstract

We study finite-horizon Markov Decision Processes with continuous states, finite actions, and a static Average Value-at-Risk (AVaR) objective. The augmented-state formulation enables dynamic programming but introduces an additional continuous state variable, making exact solutions computationally expensive. We propose a probabilistic Approximate Dynamic Programming (ADP) method using Gaussian processes to approximate Q-functions in the difficult region of the augmented state space, while exploiting risk-neutral continuation elsewhere. Bellman backups are allocated using acquisition scores based on decision uncertainty, reachability, and tail relevance. We also derive approximation and policy-performance error bounds. Numerical experiments show that decision-aware sampling often achieves better policy performance per Bellman backup than random or uncertainty-based sampling. However, incorporating reachability or tail relevance does not consistently improve performance, and greater backup efficiency does not necessarily imply lower runtime. These results suggest that decision-aware Bellman allocation is a promising approach to computationally efficient risk-averse control.

1 Introduction

Traditional stochastic control often minimizes the expected cumulative cost. However, in applications where large losses are particularly meaningful, the expected cost may not adequately represent the risk of severe outcomes. Average Value-at-Risk (AVaR), also known as Conditional Value-at-Risk (CVaR) or Expected Shortfall (ES), provides an alternative by focusing on the upper tail of the cost distribution. The optimization representation of AVaR introduced by Rockafellar and Uryasev [2002] makes it possible to formulate risk-averse control problems in terms of expected excess costs.

For Markov Decision Processes (MDPs), minimizing the static AVaR of cumulative cost is challenging because the objective does not directly admit standard Bellman recursion. Bäuerle and Ott [2011] addressed this problem using an augmented state that tracks the accumulated cost relative to a risk threshold. Although this formulation enables dynamic programming, it introduces an additional state dimension, making exact solutions computationally expensive for continuous-state problems.

Approximate Dynamic Programming (ADP) can reduce this computational cost, but its performance depends on how the available computational effort is allocated. In particular, approximating

value functions accurately across the entire state space may be unnecessary when the main objective is to obtain a good policy. Approximation errors do not necessarily change the optimal action, as discussed through the action-gap phenomenon by Farahmand [2011]. This motivates selecting Bellman updates according to their potential effect on policy decisions.

In this paper, we investigate a probabilistic ADP method based on Gaussian Processes (GPs) for finite-horizon static AVaR control. We exploit the risk-neutral structure of the augmented state space when the accumulated cost exceeds the threshold and use GP approximations in the remaining region. To allocate Bellman backups, we consider acquisition scores based on uncertainty about the optimal action, state reachability, and relevance to the AVaR tail. We also derive approximation and policy-performance error bounds, including the effects of the AVaR confidence level and outer threshold optimization.

We evaluate the method against exact dynamic programming, a baseline ADP algorithm, and alternative GP sampling strategies. The experiments show that decision-aware sampling often achieves better policy performance per Bellman backup than random or uncertainty sampling. However, adding reachability or tail weighting does not consistently improve the results, and backup efficiency does not necessarily imply lower runtime. These findings motivate further investigation of decision-aware allocation of computational effort in risk-averse control.

2 Related work

2.1 AVaR control and approximate dynamic programming

The optimization representation of AVaR introduced by Rockafellar and Uryasev [2002] provides a foundation for risk-averse stochastic optimization. For Markov decision processes, applying AVaR to the cumulative cost generally prevents direct Bellman recursion on the original state space. Bäuerle and Ott [2011] addressed this problem by introducing an augmented state that tracks the remaining risk threshold. Their formulation also establishes the affine structure of the value function after the threshold has been crossed. Under nonnegative costs, this corresponds to the risk-neutral continuation used in this paper.

Several other approaches have been developed for risk-averse sequential decision making. Chow et al. [2015] proposed an augmented dynamic programming formulation based on the dual representation of AVaR, together with approximate value iteration. Jiang and Powell [2018] studied risk-averse approximate dynamic programming with quantile-based risk measures and importance sampling. However, their recursively defined risk criteria differ from the static AVaR of cumulative cost considered here. More recently, Muni et al. [2026] developed an alternative Bellman formulation for static AVaR control based on reward redistribution.

These methods demonstrate that risk-averse control can be addressed through different dynamic programming formulations and approximation techniques. In the present work, we follow the augmented-state approach of Bäuerle and Ott and exploit its established structural properties. Our focus is on approximating the remaining difficult region efficiently by allocating a limited number of Bellman backups to selected states.

2.2 Gaussian processes and decision-aware sampling

Gaussian processes have been widely used for value-function approximation in reinforcement learning. Engel et al. [2005] developed Gaussian-process methods for value and action-value estimation, while Grande et al. [2014] studied sample-efficient reinforcement learning using GPs. Unlike these approaches, our method uses GPs to approximate Q-functions from model-based Bellman targets

in a backward finite-horizon recursion. The GP predictions provide both approximate Q-values and uncertainty estimates, which we use to guide the selection of Bellman updates.
Selective allocation of computational effort is also an established idea in approximate dynamic programming. Wingate and Seppi [2005] studied prioritization methods for Bellman updates, while Ryzhov et al. [2019] developed a Bayesian exploration framework for approximate dynamic programming. These methods show that computational resources can be allocated according to the expected usefulness of additional information. Our approach follows a similar principle but focuses specifically on uncertainty about the optimal action rather than value-function accuracy alone.
The motivation for decision-aware sampling is closely related to the action-gap phenomenon studied by Farahmand [2011], which explains why approximation errors do not necessarily change the optimal action. A related approach was developed by Cakmak et al. [2024], who used Gaussian processes for contextual ranking and selection under a limited simulation budget. Our decision score follows the same general idea by measuring uncertainty in the ordering of the two actions with the lowest predicted Q-values. We adapt this criterion to select Bellman backups in the difficult region of an augmented static-AVaR MDP. We also investigate reachability and tail relevance, which are related to importance sampling in risk-averse ADP [Jiang and Powell, 2018] and tail-focused sampling for AVaR evaluation [Bourigault et al., 2026].

3 Problem formulation

3.1 Finite-horizon Markov decision process

Consider a finite-horizon controlled Markov process with time indices $t = 0, 1, \ldots, T$, continuous state space $\mathcal{X} \subseteq \mathbb{R}$, and finite action space $\mathcal{A}$. The system's dynamics are

$$x_{t+1} = f(x_t, a_t, \xi_t) \tag{1}$$

where

- $x_t \in \mathcal{X}$ denotes the state at time t,
- $a_t \in \mathcal{A}$ is the selected action, and
- $\{\xi_t\}_{t=0}^{T-1}$ is a sequence of independent and identically distributed random disturbances.

The initial state $x_0 \in \mathcal{X}$ is assumed to be fixed.
Let the cumulative cost induced by a policy π be defined as

$$C^{\pi} = \sum_{t=0}^{T-1} c_t(x_t, a_t) + c_T(x_T), \tag{2}$$

where

- $c_t : \mathcal{X} \times \mathcal{A} \to \mathbb{R}_{\geq 0}$ is the stage cost at time t, and
- $c_T : \mathcal{X} \to \mathbb{R}_{\geq 0}$ is the terminal cost, depending only on the terminal state.

We assume that the dynamics and cost functions are measurable and that the cumulative cost is integrable under every admissible policy.

3.2 AVaR objective

For confidence level $\alpha \in (0,1)$, the objective is

$$\min_{\pi} \mathrm{AVaR}_{\alpha}(C^{\pi}).$$

Using the Rockafellar and Uryasev [2002] representation,

$$\mathrm{AVaR}_{\alpha}(X) = \min_{\eta \in \mathbb{R}} \left[\eta + \frac{1}{1-\alpha} \mathbb{E}\left[(X-\eta)_{+} \right] \right]. \tag{3}$$

Hence the control problem can be written as

$$\begin{aligned} \min_{\pi} \mathrm{AVaR}_{\alpha}(C^{\pi}) &= \min_{\pi} \min_{\eta \in \mathbb{R}} \left[\eta + \frac{1}{1-\alpha} \mathbb{E}[(C^{\pi}-\eta)_{+}] \right] \\ &= \min_{\eta \in \mathbb{R}} \left\{ \eta + \frac{1}{1-\alpha} \min_{\pi} \mathbb{E}[(C^{\pi}-\eta)_{+}] \right\} \end{aligned} \tag{4}$$

Since the ramp function acts on the cumulative cost, the problem generally cannot be solved using standard Bellman recursion on the original state space.

3.3 Augmented-state formulation

To make dynamic programming tractable, we introduce an augmented state, according to Bäuerle and Ott [2011]:

$$y_t = \sum_{s=0}^{t-1} c_s(x_s, a_s) - \eta, \tag{5}$$

or, equivalently,

$$y_{t+1} = y_t + c_t(x_t, a_t), \quad y_0 = -\eta \tag{6}$$

The terminal value function then becomes

$$V_T(x,y) = (y + c_T(x))_{+}. \tag{7}$$

For $t < T$, we can define

$$Q_t(x,y,a) = \mathbb{E}\left[V_{t+1}\left(x_{t+1}, y_{t+1}\right) \mid x_t = x, y_t = y, a_t = a \right] \tag{8}$$

and

$$V_t(x,y) = \min_{a \in \mathcal{A}} Q_t(x,y,a). \tag{9}$$

The optimal fixed-η value for the entire horizon is therefore

$$V_0(x_0, -\eta). \tag{10}$$

The corresponding complete AVaR objective is

$$J(\eta) = \eta + \frac{1}{1-\alpha} V_0(x_0, -\eta), \tag{11}$$

and

$$\eta^{\star} \in \arg\min_{\eta} J(\eta).$$

3.4 Risk-neutral region of the augmented state space

Once $y_t \geq 0$ at some time step t, under nonnegative stage and terminal costs,

$$y_s = \underbrace{y_t}_{\geq 0} + \sum_{u=t}^{s-1} \underbrace{c_u(x_u, a_u)}_{\geq 0}, \quad t < s \leq T$$

Thus, the ramp function $(\cdot)_+$ does not change the objective of the optimization, meaning that

$$Q_t(x, y, a) = y + Q_t^{\mathrm{RN}}(x, a), \tag{12}$$

and the optimal action remains identical

$$\arg\min_{a \in \mathcal{A}} Q_t(x, y, a) = \arg\min_{a \in \mathcal{A}} Q_t^{\mathrm{RN}}(x, a), \tag{13}$$

where Q_t^{RN} is a risk-neutral Q-function corresponding to the following control optimization problem

$$Q_t^{\mathrm{RN}}(x, a) = c_t(x, a) + \mathbb{E}\left[V_{t+1}^{\mathrm{RN}}(x_{t+1}) \mid x_t = x, a_t = a\right], \tag{14}$$

and

$$V_t^{\mathrm{RN}}(x) = \min_{a \in \mathcal{A}} Q_t^{\mathrm{RN}}(x, a), \quad V_T^{\mathrm{RN}}(x) = c_T(x). \tag{15}$$

3.5 Risk-neutral Q-function approximation

We approximate the risk-neutral Q-function using backward dynamic programming with regression. Starting from the terminal condition

$$V_T^{\mathrm{RN}}(x) = c_T(x),$$

the Q-function and its approximation at $t = T - 1$ are given by

$$\hat{Q}_{T-1}^{\mathrm{RN}} \approx Q_{T-1}^{\mathrm{RN}}(x, a) = c_{T-1}(x, a) + \mathbb{E}\left[c_T(x_T) \mid x_{T-1} = x, a_{T-1} = a\right].$$

The corresponding value function approximation is

$$\hat{V}_{T-1}^{\mathrm{RN}}(x) = \min_{a \in \mathcal{A}} \hat{Q}_{T-1}^{\mathrm{RN}}(x, a).$$

This approximation is then used to construct the Bellman targets at the preceding time step,

$$\hat{Q}_t^{\mathrm{RN}}(x, a) \approx c_t(x, a) + \mathbb{E}\left[\hat{V}_{t+1}^{\mathrm{RN}}(x_{t+1}) \mid x_t = x, a_t = a\right], \tag{16}$$

$$\hat{V}_t^{\mathrm{RN}}(x) = \min_{a \in \mathcal{A}} \hat{Q}_t^{\mathrm{RN}}(x, a).$$

The procedure is repeated backward from $t = T - 1$ to $t = 0$.

3.6 Computational challenge

The state augmentation allows us to use dynamic programming, making the problem tractable. However, the problem is now formulated over $\mathcal{X}\times\mathbb{R}$, leaving us with two continuous states to handle. This makes the direct Bellman solution very expensive, especially as horizon and discretization increase.

The risk-neutral region $y \geq 0$ reduces this difficulty by allowing us to use risk-neutral continuation approximations. The remaining $y < 0$ region still requires the calculation (or approximation) of the augmented-state value function.

Under a limited computational budget, evaluating the Bellman operator uniformly across this region may be inefficient as different states have different effects on policy and resulting AVaR objective. This motivates the development of a sampling strategy that allocates Bellman evaluations to states where they are most relevant to the accuracy of the solution.

4 Methodology

4.1 Exact dynamic programming

Both the proposed and baseline algorithms can be verified against an exact dynamic programming solution if the horizon T is small. The expectation in (8) is computed using Riemann summation

$$\begin{aligned}\mathbb{E}\left[V_{t+1}(x_{t+1}, y_{t+1}) \mid x_t = x, y_t = y, a_t = a\right] &= \int V_{t+1}(x_{t+1}, y_{t+1}) p_{x_{t+1}}\left(x_{t+1} \mid x_t = x, a_t = a\right) dx_{t+1} \\ &= \int_{N_\xi} V_{t+1}(f(x_t, a_t, \xi_t), y_t + c_t(x_t, a_t))\ p_\xi(\xi_t)\ d\xi_t \\ &\approx \sum_{i=0} V_{t+1}(f(x_t, a_t, \xi^{(i)}), y_t + c_t(x_t, a_t))\ w_i\end{aligned} \tag{17}$$

where

- p_ξ is the density of disturbance distribution,
- $\xi^{(i)}$ are discretization points of the disturbance domain, defined by:
  - $\xi^{(0)} = \xi_{\min}$,
  - $\xi^{(N_\xi)} = \xi_{\max}$,
  - $\xi^{(i+1)} = \xi^{(i)} + \Delta_\xi$,
  - $\Delta_\xi = \frac{\xi_{\max} - \xi_{\min}}{N_\xi}$,
- $w_i = \frac{p_\xi(\xi^{(i)})}{\sum_j p_\xi(\xi^{(j)})}$ is the normalized discrete weights of points.

For a sufficiently wide integration domain and fine discretization mesh, the numerical approximation is expected to approach the true expectation. In this paper we refer to this method as *Exact Dynamic Programming* to distinguish it from the approximate dynamic programming methods introduced in the following subsections. Here, *exact* means that the Bellman recursion is solved directly, without approximating the value functions.

4.2 Baseline approximate dynamic programming

The exact dynamic programming algorithm serves as a reliable reference for small horizons, but its computational cost can grow exponentially with the horizon. To reduce computational cost, we use an Approximate Dynamic Programming (ADP) algorithm.

The method is based on Monte Carlo rollouts. To initialize the value function at an unseen state (t, x, y), we simulate N trajectories from that state. The actions in the trajectories are chosen using the approximate risk-neutral policy, which provides a computationally cheap way to estimate the remaining cumulative cost. For each trajectory, we calculate the remaining cumulative cost and apply the ramp function. The initial estimate is then obtained by averaging these values.

To improve the estimate, we also use the risk-neutral value approximation and define

$$\hat{V}_t^{\text{init}}(x, y) = \max\left\{\frac{1}{N}\sum_{i=1}^{N}(y + C_{t:T}^{(i)})_+,\ \left(y + \hat{V}_t^{\text{RN}}(x)\right)_+\right\}, \tag{18}$$

where $C_{t:T}^{(i)}$ is the remaining cumulative cost of the i-th trajectory.

In the equation above we have used Jensen's bound: given that the ramp function is convex, we can obtain

$$\begin{aligned} V_t(x, y) &= \min_{\pi} \mathbb{E}^{\pi}\left[(y + C_{t:T}^{\pi})_+\right] \\ &\geq \min_{\pi}\left(\mathbb{E}^{\pi}\left[y + C_{t:T}^{\pi}\right]\right)_+ \\ &= \left(y + V_t^{\text{RN}}(x)\right)_+ \end{aligned}$$

Since the approximation of the risk-neutral value function is used we cannot guarantee that the right-hand side is a valid lower bound. However, using the risk-neutral approximation gave us a useful proxy to a lower bound for the current value function. Monte Carlo estimation was compared to the risk-neutral value function and their maximum was used as the initial value.

At terminal states, we use the exact terminal value. For visited nonterminal states, we apply the Bellman updates described in Section 3.3, using the initialized values whenever necessary. The initialized values are stored and reused to avoid repeated simulations.

For $y \geq 0$, we can use approximation of the risk-neutral Q-function as discussed in Section 3.5. In this paper we use linear regression to approximate each Q-function:

$$\hat{Q}_t^{\text{RN}}(x, a) = \phi^T(x)\theta_{t,a} \approx Q_t^{\text{RN}}(x, a) \tag{19}$$

where

- $\phi(x) : \mathcal{X} \to \mathbb{R}^d$ is a feature map,
- $\theta_{t,a} \in \mathbb{R}^d$ is a vector of regression parameters.

The feature map combines quadratic and radial basis function (RBF) features:

$$\phi(x) = \begin{bmatrix}1 & z & z^2 & \psi_1(x) & \cdots & \psi_{d-3}(x)\end{bmatrix}^\top, \tag{20}$$

where $z \in [-1, 1]$ is obtained by linearly scaling x, and

$$\psi_i(x) = \exp\left(-\frac{(x - c_i)^2}{2\ell^2}\right), \quad i = 1, \ldots, d-3.$$

The RBF centers c_i are uniformly spaced over a slightly enlarged interval around $[x_{\min}, x_{\max}]$.

The model training process is recursive as described in Section 3.5.

To reuse previously computed values, we store them in a lookup table. Since the state variables x and y are continuous, we round them to six decimal places before constructing the cache keys. This allows sufficiently close states to share the same stored value, reducing the number of repeated computations at the cost of a small additional approximation.
The full algorithm can be found in Appendix A.
The baseline ADP reduces computational cost by exploiting the risk-neutral region, reusing previously computed values, and approximating the values of unseen states using Monte Carlo rollouts instead of full Bellman recursion. However, its accuracy depends on which states are visited during training, and the algorithm does not explicitly account for the importance of individual Bellman updates.

4.3 Proposed probabilistic ADP

The baseline ADP reduces computational cost, but it does not explicitly account for the importance of states when performing Bellman updates. As discussed in Section 3.6, different states may have different effects on the resulting policy and AVaR objective. To address this, we propose a probabilistic ADP method that allocates Bellman evaluations to selected states in the region $y < 0$, while continuing to use the risk-neutral approximation for $y \geq 0$.
The proposed algorithm uses Gaussian Processes (GPs) to approximate the Q-functions, providing both point predictions of the value and estimates of uncertainty. To decide where to perform Bellman updates, we construct sets of candidate states and evaluate them using three criteria: reachability, uncertainty about the optimal action, and relevance to the AVaR tail. These criteria guide the allocation of a limited computational budget. The GP models are then trained backward in time using the selected Bellman evaluations. Additionally, the use of GPs reduces our need to discretize the state space as we have done in the baseline ADP by rounding the states. The following subsections describe each component of the proposed method. The resulting algorithm can be found in Appendix B.

4.3.1 GP approximation of Q-functions

For the region $y < 0$, we approximate the Q-functions using GPs. Unlike the regression models used for the risk-neutral approximation, GPs provide both point predictions and estimates of uncertainty. We train a separate GP model for each time step t and action $a \in \mathcal{A}$, using the augmented state $s = (x, y)$ as input.
The training data are obtained by applying the Bellman operator at selected augmented states. For a training state $s_j = (x_j, y_j)$ and action a, the target value is calculated as

$$q_{t,a}^{(j)} = \sum_{\xi^{(i)} \in \Xi} w_i \hat{V}_{t+1} \left(f(x_j, a, \xi^{(i)}), y_j + c_t(x_j, a) \right). \tag{21}$$

where $\hat{V}_{t+1}$ is the value function approximation obtained at the next time step. The resulting training dataset for each GP is

$$\mathcal{D}_{t,a} = \left\{ \left(s_j, q_{t,a}^{(j)} \right) \right\}_{j=1}^{n_t}$$

where n_t is the number of selected training states at time t.
We fit a GP with mean function $m_{t,a}$ and covariance kernel $k_{t,a}$ to the approximate Bellman targets defined in (21).

We use an RBF kernel with fixed hyperparameters and scale the GP inputs using fixed state-space bounds. Before fitting each GP, the training targets are standardized, and predictions are transformed back to their original scale.
Given the training data, the GP provides a predictive mean $\mu_{t,a}(s)$ and variance $\sigma^2_{t,a}(s)$. These are calculated as

$$\begin{aligned} \mu_{t,a}(s) &= m_{t,a}(s) + k(s)^\top (K + \lambda I)^{-1}(q_{t,a} - m_{t,a}), \\ \sigma^2_{t,a}(s) &= k_{t,a}(s,s) - k(s)^\top (K + \lambda I)^{-1} k(s), \end{aligned} \tag{22}$$

where:

- K is the covariance matrix of the training inputs,
- $k(s)$ is the vector of covariance between training inputs and s,
- $q_{t,a}$ is the vector of training targets,
- $m_{t,a}$ is the prior mean at training inputs,
- λ is a regularizer.

We use the predictive mean to approximate the Q-functions:

$$\hat{Q}_t(x, y, a) = \mu_{t,a}(x, y), \tag{23}$$

which gives the corresponding value function and policy:

$$\begin{aligned} \hat{V}_t(x,y) &= \max\left\{ \min_{a\in\mathcal{A}} \hat{Q}_t(x,y,a),\ \left(y + \hat{V}_t^{\mathrm{RN}}(x)\right)_+ \right\}, \\ \hat{\pi}_t(x,y) &= \arg\min_{a\in\mathcal{A}} \hat{Q}_t(x,y,a). \end{aligned} \tag{24}$$

These approximations are used only for $y < 0$. For $y \geq 0$, we continue using the risk-neutral approximation described earlier, while at $t = T$ we use the exact terminal value.
The predictive uncertainty $\sigma_{t,a}(x, y)$ provides information about how uncertain the GP model is at a particular state. It measures uncertainty within the fitted model, but it does not necessarily represent the actual Q-function approximation error. We use this information when selecting states for Bellman updates, as described in the following subsections.

4.3.2 Candidate-state construction

Since the augmented state space $\mathcal{X} \times \mathbb{R}$ is continuous, we cannot evaluate the Bellman operator at every possible state. Instead, for each time step t, we construct a finite set of candidate states

$$\mathcal{S}_t = \{s_t^{(j)}\}_{j=1}^{M_t}, \qquad s_t^{(j)} = (x_t^{(j)}, y_t^{(j)}), \tag{25}$$

where M_t is the number of candidates at time t. These states are used to select the points at which Bellman updates will be performed.
The candidate states are restricted to the region $y < 0$, since for the region $y \geq 0$ we use risk-neutral approximation. Note that, since the stage costs are nonnegative, the augmented state satisfies $y_t \geq -\eta$ for a fixed threshold η.
To construct the candidate set, we consider a range of possible values of x at each time step, based on the system dynamics and disturbance distribution. We then generate candidate states from the region

$$\begin{aligned} \mathcal{R}_t &= [x_t^{\min}, x_t^{\max}] \times [-\eta, 0), \quad 0 < t < T \\ \mathcal{R}_t &= \{(x_0, -\eta)\}, \qquad\qquad\qquad\quad t = 0, \end{aligned} \tag{26}$$

where $x_t^{\min}$ and $x_t^{\max}$ define the range of states considered at time t. We then select candidate states from this region without discretizing the entire state space.
However, not every state in this region is necessarily reachable. Since y depends on x as it accumulates the costs through the trajectory, some candidate states might be unlikely.
To avoid performing Bellman updates at all candidate states, we apply a scoring system to determine which candidate states should receive more computational effort. The acquisition scores are described in the following subsection.

4.3.3 Bellman acquisition scores

The candidate set $\mathcal{S}_t$ contains possible states for Bellman updates. However, evaluating the Bellman operator at every candidate would be computationally expensive. To select the most useful states, we introduce acquisition scores based on three criteria: uncertainty about the optimal action, reachability, and relevance to the AVaR tail.

Decision uncertainty. The first criterion measures how uncertain the GP models are about the optimal action. Since our objective is to obtain an accurate policy, reducing uncertainty about the optimal action may be more important than reducing the approximation error of every Q-function. For a candidate state $s = (x, y)$, let a_1 and a_2 denote the actions with the smallest and second-smallest GP predictive means, respectively. We define the predicted action gap as

$$\Delta_t(s) = \mu_{t,a_2}(s) - \mu_{t,a_1}(s). \tag{27}$$

A small gap indicates that the two actions have similar predicted values. However, the gap alone does not tell us how confident the GP models are about their ordering.
The GP predictions are Gaussian. Assuming that the GP predictions for different actions are independent, their difference is also Gaussian. Using this fact we can estimate the probability of the second action being better than the first one. Let us first define their difference as a Gaussian

$$\begin{aligned} Q_1 &\sim \mathcal{N}(\mu_{t,a_1}(s), \sigma^2_{t,a_1}(s)) \\ Q_2 &\sim \mathcal{N}(\mu_{t,a_2}(s), \sigma^2_{t,a_2}(s)) \\ Z_t(s) &= Q_2 - Q_1 \\ &\sim \mathcal{N}(\Delta_t(s), \sigma^2_{t,a_1}(s) + \sigma^2_{t,a_2}(s)) \end{aligned}$$

Now we can define a decision score as

$$\begin{aligned} D_t(s) &= 2\mathbb{P}\left(Z_t(s) < 0\right) \\ &= 2\Phi\left(-\frac{\Delta_t(s)}{\sqrt{\sigma^2_{t,a_1}(s) + \sigma^2_{t,a_2}(s)}}\right) \end{aligned} \tag{28}$$

where Φ is the standard normal cumulative distribution function (CDF). The factor of 2 normalizes the score to the range $[0, 1]$.
Larger values of $D_t(s)$ show greater uncertainty about the ordering of the two actions. This score is a proxy for decision uncertainty rather than the exact probability of selecting the wrong action, since other actions are not considered in the comparison.
We also define an uncertainty score

$$U_t(s) = \max_{a \in \mathcal{A}} \sigma_{t,a}(s) \tag{29}$$

which measures the largest predictive standard deviation among the Q-function GP models, regardless of their predicted values.

Reachability. The second criterion measures how close a candidate state is to states visited during simulated trajectories. Bellman updates at states that are unlikely to be visited may have little effect on the resulting policy.
To estimate reachability, we simulate trajectories under a selected policy and collect the visited states at each time step. For each candidate s, we then find its k nearest neighbors among the rollout states.
Since x and y can have different scales and are correlated, we use the Mahalanobis distance:

$$d_t(s, s') = \sqrt{(s - s')^T (\Sigma_t + \varepsilon I)^{-1} (s - s')}, \tag{30}$$

where Σ_t is the covariance matrix of the rollout states at time t, and $\varepsilon > 0$ is a small regularization parameter.
We then define a reachability score $R_t(s)$ using a distance from a state s to its k-th nearest neighbor $\tilde{s}^{(k)}$:

$$R_t(s) = \frac{1}{d_t(s, \tilde{s}^{(k)})^2 + \delta} \tag{31}$$

with $\delta > 0$ being a constant that prevents division by zero. The scores are then normalized to the range $[0, 1]$ by dividing each over the maximal score across the candidate states.
This is not the only way to define reachability. One can also consider average distances to nearest neighbors, state visitation probabilities or densities, or other measures of reachability.

Tail relevance. AVaR depends on the tail of the total cost distribution, so we want to give more attention to the states that are more relevant to the distribution tail.
Recall that the terminal value function is

$$V_T(x, y) = (y + c_T(x))_+.$$

Therefore, a trajectory contributes a positive value to the fixed-η objective only when its total cost exceeds η. Thus, the score measures relevance to costs exceeding the chosen threshold η, which may differ from the actual AVaR tail.
Using simulated trajectories, we can estimate the tail relevance of a candidate state from the outcomes of nearby rollout states. One possible score is

$$L_t(s) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(s)} \mathbb{I}\left\{ y_T^{(i)} + c_T(x_T^{(i)}) > 0 \right\}, \tag{32}$$

where $\mathcal{N}_k(s)$ now denotes the indices of the nearest rollout states, and $(x_T^{(i)}, y_T^{(i)})$ is the terminal state of the corresponding trajectory.
This score estimates how frequently trajectories passing near a candidate end up in the AVaR tail. It is based on the rollout policy and does not necessarily represent the tail probability under the optimal policy.

Combination of acquisition scores. The described criteria provide different information about the usefulness of a Bellman update. Decision uncertainty identifies states where the optimal action is unclear, reachability favors states that are likely to be visited, and tail relevance favors states associated with larger cumulative costs.
We can use these scores separately or combine them into a single acquisition function:

$$A_t(s) = F\left(D_t(s), R_t(s), L_t(s)\right), \tag{33}$$

where F denotes the selected combination rule. Different constructions of $A_t(s)$ result in different policies, giving us a field to experiment on.

4.3.4 Backward training algorithm

We train the GP models backward in time, starting from $t = T - 1$ and continuing to $t = 1$. The exact terminal value function is used to calculate the Bellman targets at $t = T - 1$.
For acquisition methods involving reachability or tail relevance, we first simulate trajectories under the approximate risk-neutral policy. The visited states and terminal costs are then used to calculate the corresponding acquisition scores.
At each time step, we randomly select N_{init} states from the candidate set $\mathcal{S}_t$. For each selected state, we calculate the Bellman targets for all actions and fit the initial GP models.
We then calculate acquisition scores for the remaining candidate states using the fitted GP models. The states with the highest scores are selected in batches. After each batch, we calculate the new Bellman targets, add them to the training data, and refit the GP models. This continues until the computational budget for the current time step is reached. For random sampling, all training states are selected randomly and the GP models are fitted only once.
The Bellman targets are calculated using the value-function approximation at the next time step. For $y < 0$, we use the GP prediction together with the risk-neutral approximation and the lower bound of zero. For $y \geq 0$, we use the risk-neutral continuation, while at $t = T$ we use the exact terminal value.
After training the GP models for $t = 1, \ldots, T - 1$, we calculate the Q-values at the initial state $(x_0, -\eta)$ directly using the Bellman operator. Therefore, no GP model is needed at $t = 0$. The resulting value is then used in the outer optimization over η, discussed in the following subsection.

4.4 Outer optimization over η

Recall the optimization problem (4):

$$\min_{\pi} \text{AVaR}_{\alpha}(C^{\pi}) = \min_{\eta \in \mathbb{R}} \left\{ \eta + \frac{1}{1-\alpha} \min_{\pi} \mathbb{E}[(C^{\pi} - \eta)_+] \right\}$$

In the previous subsections we have formulated baseline ADP and probabilistic ADP methods to solve the inner optimization problem for fixed η. In this subsection we will discuss the outer optimization problem over the threshold η. Consider a finite set of candidate thresholds

$$\mathcal{H} = \{\eta_1, \eta_2, \ldots, \eta_{N_\eta}\}$$

over which to search for an optimal threshold.
Then, we modify the candidate domain stated in (26) as

$$\mathcal{R}_t = [x_t^{\min}, x_t^{\max}] \times [-\max \mathcal{H}, 0), \quad 0 < t < T. \tag{34}$$

This allows the same GP models to be used for all candidate thresholds.

Baseline ADP While we cannot modify the baseline ADP solution to account for several values of η at the same time, we can use a shared cache $\mathcal{C}$ to reuse the already calculated value functions. After calculating all of the values $\hat{V}_0(x_0, -\eta)$ for $\eta \in \mathcal{H}$ we can define the optimal approximate η as

$$\hat{\eta} = \arg\min_{\eta in \mathcal{H}} \hat{J}(\eta) = \arg\min_{\eta \in \mathcal{H}} \left\{ \eta + \frac{1}{1-\alpha} \hat{V}_0(x_0, -\eta) \right\}. \tag{35}$$

Probabilistic ADP For probabilistic ADP, we can avoid training separate GP models for each $\eta \in \mathcal{H}$. Instead, we train a single collection of GP models for $t = 1, \ldots, T-1$, using candidate states that cover the relevant range of augmented states for all thresholds in $\mathcal{H}$. We then evaluate the Bellman operator at $(x_0, -\eta)$ for each candidate threshold, without training a GP model at $t = 0$. The resulting values $\hat{V}_0(x_0, -\eta)$ are used to calculate the corresponding objectives $\hat{J}(\eta)$ and select the best threshold. Each evaluation at the initial state requires one Bellman backup per action and is included in the computational budget.
The reuse of the training GPs is possible because the threshold enters the augmented problem only through the initial condition $y_0 = -\eta$. The Bellman recursion at a given augmented state does not otherwise depend on η. The fitted continuation models can therefore be shared across root queries, provided that their training domain covers the required augmented states.
To conclude the section, we have stated an exact dynamic programming solution to compare the approximate methods against. The exact DP algorithm estimates the expectation using Riemann summation. It is computationally expensive, but can serve as a reliable benchmark for the approximate algorithm. The baseline ADP algorithm was designed using Monte Carlo rollouts and value initialization at unseen states. The main focus of the section is the proposed probabilistic ADP algorithm, which performs Q-function approximation using Gaussian processes and allocates Bellman backups to the candidate states using scoring based on decision uncertainty, reachability, and tail relevance of the candidates. At the end of the section we have briefly discussed an outer optimization over the threshold η for baseline and probabilistic ADP algorithms.

5 Theoretical analysis

5.1 Preliminaries and assumptions

We consider the finite-horizon MDP and its augmented-state formulation introduced in Section 3. Throughout this section, V_t and $\hat{V}_t$ denote the exact and approximate augmented-state value functions, respectively. Similarly, V_t^{RN} and $\hat{V}_t^{\mathrm{RN}}$ denote their risk-neutral counterparts.
The exact value and Q-functions are defined with respect to the original continuous disturbance distribution. The local approximation errors also account for numerical integration errors introduced by the algorithms.
We make the following assumptions:

1. **Nonnegative costs.** The stage and terminal costs are nonnegative

$$c_t(x, a) \geq 0, \qquad c_T(x) \geq 0.$$

2. **Well-defined values.** All expectations appearing in the Bellman equations are finite on the states considered in the analysis.

3. **Threshold domain.** The outer optimization is restricted to a compact interval $\mathcal{I}_\eta \subset \mathbb{R}$, assumed to contain at least one global optimal threshold η^*. The numerical optimization uses a finite candidate set $\mathcal{H} \subset \mathcal{I}_\eta$.

For each t, let $\mathcal{Z}_t$ denote the set of augmented states (x, y) reachable from $(x_0, -\eta)$, with $\eta \in \mathcal{I}_\eta$, under admissible action sequences and noise realizations.
We partition this domain into

$$\mathcal{Z}_t^+ = \{(x, y) \in \mathcal{Z}_t : \ y \geq 0\} \quad \text{and} \quad \mathcal{Z}_t^- = \{(x, y) \in \mathcal{Z}_t : \ y < 0\} \tag{36}$$

For any function f defined on a domain D, we use the supremum norm

$$\|f\|_{\infty,D} = \sup_{z\in D} |f(z)|.$$

We define the approximation errors in the two regions as

$$\epsilon_t^+ = \|\hat{V}_t - V_t\|_{\infty,\mathcal{Z}_t^+}, \qquad \epsilon_t^- = \|\hat{V}_t - V_t\|_{\infty,\mathcal{Z}_t^-}. \tag{37}$$

We assume that the local approximation errors defined below are uniformly bounded over their respective domains. These domains may be unbounded, and such bounds are not guaranteed by GP regression. Therefore, the theoretical results are conditional on this assumption. For an empty domain we take the error to be zero.
The resulting bounds are deterministic and conditional on the fitted approximation functions; no probabilistic guarantees on GP approximation errors are assumed.
By the risk-neutral decomposition and the approximation rule used by the proposed method,

$$V_t(x,y) = y + V_t^{\mathrm{RN}}(x), \qquad \hat{V}_t(x,y) = y + \hat{V}_t^{\mathrm{RN}}(x). \tag{38}$$

Consequently,

$$\epsilon_t^+ = \sup_{x\in\mathcal{X}_t^+} \left|\hat{V}_t^{\mathrm{RN}}(x) - V_t^{\mathrm{RN}}(x)\right|,$$

where

$$\mathcal{X}_t^+ = \{x : \ \exists y \geq 0, (x,y) \in \mathcal{Z}_t\}$$

Since the proposed method uses the exact terminal value function, we have

$$\epsilon_T^+ = \epsilon_T^- = 0. \tag{39}$$

5.2 Augmented-state approximation error

5.2.1 Risk-neutral region ($y \geq 0$)

Let us recall and define the main quantities that will be used in this paragraph

- Risk-neutral Q-function (14)

$$Q_t^{\mathrm{RN}}(x,a) = c_t(x,a) + \mathbb{E}\left[V_{t+1}^{\mathrm{RN}}(x_{t+1}) \mid x_t = x, a_t = a\right]$$

- Bellman target for Q-function approximation using approximate value function

$$\tilde{Q}_t^{\mathrm{RN}}(x,a) = c_t(x,a) + \mathbb{E}\left[\hat{V}_{t+1}^{\mathrm{RN}}(x_{t+1}) \mid x_t = x, a_t = a\right]$$

- Recursive Q-function approximation (16)

$$\hat{Q}_t^{\mathrm{RN}}(x,a) \approx \tilde{Q}_t^{\mathrm{RN}}(x,a)$$

- Local approximation error

$$\beta_t^{\mathrm{RN}} = \sup_{\substack{x\in\mathcal{X}_t^+ \\ a\in\mathcal{A}}} \left|\hat{Q}_t^{\mathrm{RN}}(x,a) - \tilde{Q}_t^{\mathrm{RN}}(x,a)\right|$$

Therefore, the one-step risk-neutral approximation bound is

$$\begin{aligned}\left|\hat{Q}_t^{\mathrm{RN}}(x,a) - Q_t^{\mathrm{RN}}(x,a)\right| &\leq \underbrace{\left|\hat{Q}_t^{\mathrm{RN}}(x,a) - \tilde{Q}_t^{\mathrm{RN}}(x,a)\right|}_{\leq \beta_t^{\mathrm{RN}}} + \underbrace{\left|\tilde{Q}_t^{\mathrm{RN}}(x,a) - Q_t^{\mathrm{RN}}(x,a)\right|}_{\leq \epsilon_{t+1}^+} \\ &\leq \beta_t^{\mathrm{RN}} + \epsilon_{t+1}^+\end{aligned} \tag{40}$$

The first term on the right-hand side of (40) is bounded by the local approximation error by definition of β_t^{RN}, the second part is justified below:

$$\begin{aligned}\left|\tilde{Q}_t^{\mathrm{RN}}(x,a) - Q_t^{\mathrm{RN}}(x,a)\right| &\leq \left|\mathbb{E}\left[\hat{V}_{t+1}^{\mathrm{RN}}(x_{t+1}) \mid x_t = x, a_t = a\right] - \mathbb{E}\left[V_{t+1}^{\mathrm{RN}}(x_{t+1}) \mid x_t = x, a_t = a\right]\right| \\ &= \left|\mathbb{E}\left[\hat{V}_{t+1}^{\mathrm{RN}}(x_{t+1}) - V_{t+1}^{\mathrm{RN}}(x_{t+1}) \mid x_t = x, a_t = a\right]\right| \\ &\leq \mathbb{E}\left[\underbrace{\left|\hat{V}_{t+1}^{\mathrm{RN}}(x_{t+1}) - V_{t+1}^{\mathrm{RN}}(x_{t+1})\right|}_{\leq \epsilon_{t+1}^+} \mid x_t = x, a_t = a\right] \\ &\leq \epsilon_{t+1}^+,\end{aligned}$$

the second-to-last inequality is due to Jensen's inequality. The final inequality follows because for any $x \in \mathcal{X}_t^+$ and $a \in \mathcal{A}$, the corresponding successor states belong to $\mathcal{X}_{t+1}^+$.
The corresponding error in the value function is, then

$$\begin{aligned}\left|\hat{V}_t^{\mathrm{RN}}(x) - V_t^{\mathrm{RN}}(x)\right| &= \left|\min_{a\in\mathcal{A}} \hat{Q}_t^{\mathrm{RN}}(x,a) - \min_{a\in\mathcal{A}} Q_t(x,a)\right| \\ &\leq \max_{a\in\mathcal{A}} \left|\hat{Q}_t^{\mathrm{RN}}(x,a) - Q_t^{\mathrm{RN}}(x,a)\right| \\ &\leq \beta_t^{\mathrm{RN}} + \epsilon_{t+1}^+.\end{aligned}$$

Since the inequality above holds for any $x \in \mathcal{X}_t^+$, we can deduce

$$\epsilon_t^+ \leq \beta_t^{\mathrm{RN}} + \epsilon_{t+1}^+$$

Using the fact that we use the exact terminal value at time T (stated in (39)) and the recursive inequality above we can conclude

$$\epsilon_t^+ \leq \sum_{s=t}^{T-1} \beta_s^{\mathrm{RN}} \tag{41}$$

5.2.2 Difficult region ($y < 0$)

As in the previous case, we define the quantities needed for the analysis:

- Risk-sensitive Q-function (8)

$$Q_t(x,y,a) = \mathbb{E}\left[V_{t+1}(x_{t+1}, y_{t+1}) \mid x_t = x, y_t = y, a_t = a\right]$$

- Bellman target for the Q-function

$$\tilde{Q}_t(x,y,a) = \mathbb{E}\left[\hat{V}_{t+1}(x_{t+1}, y_{t+1}) \mid x_t = x, y_t = y, a_t = a\right]$$

- Q-function approximation using GP predictive mean (23) for $0 < t < T$

$$\hat{Q}_t(x, y, a) \approx \tilde{Q}_t(x, y, a).$$

- Local approximation error

$$\beta_t^- = \sup_{\substack{(x,y)\in\mathcal{Z}_t^- \\ a\in\mathcal{A}}} \left|\hat{Q}_t(x, y, a) - \tilde{Q}_t(x, y, a)\right|$$

- Risk-neutral approximation error for $(x, y) \in \mathcal{Z}_t^-$

$$\rho_t^- = \sup_{(x,y)\in\mathcal{Z}_t^-} \left|(y + \hat{V}_t^{\mathrm{RN}}(x))_+ - (y + V_t^{\mathrm{RN}}(x))_+\right|.$$

At $t = 0$, the approximate Q-function is evaluated directly as

$$\hat{Q}_0(x_0, -\eta, a) = \sum_{i=0}^{N_\xi} w_i \hat{V}_1\left(f(x_0, a, \xi^{(i)}), -\eta + c_0(x_0, a)\right).$$

Therefore, β_0^- contains only the numerical integration error relative to the continuous Bellman target $\tilde{Q}_0$ and does not contain GP regression error. The corresponding value function is calculated as

$$\hat{V}_0(x, -\eta) = \min_{a\in\mathcal{A}} \hat{Q}_0(x, -\eta, a),$$

while the value functions for states $t = 1, \ldots, T-1$ are obtained using (24).
The risk-neutral error ϵ_t^+ is defined only over $\mathcal{X}_t^+$, whereas the risk-neutral approximation is also used for states with $y < 0$. Since their physical states x may lie outside $\mathcal{X}_t^+$, we account for the corresponding proxy error separately through ρ_t^-.
Since $y_{t+1} = y + c_t(x_t, a_t)$, the continuation error depends on whether the next state belongs to the risk-neutral or difficult region. Thus,

$$\left|\tilde{Q}_t(x, y, a) - Q_t(x, y, a)\right| \le \begin{cases} \epsilon_{t+1}^+, & y + c_t(x, a) \ge 0 \\ \epsilon_{t+1}^-, & y + c_t(x, a) < 0 \end{cases}$$

or, simpler,

$$\left|\tilde{Q}_t(x, y, a) - Q_t(x, y, a)\right| \le \max\{\epsilon_{t+1}^+, \epsilon_{t+1}^-\}.$$

Recall that we bound the value function approximation as

$$\hat{V}_t(x, y) = \max\left\{\min_{a\in\mathcal{A}} \hat{Q}_t(x, y, a), \left(y + \hat{V}_t^{\mathrm{RN}}(x)\right)_+\right\}.$$

Let

$$B_t(x, y) = (y + V_t^{\mathrm{RN}}(x))_+, \quad \hat{B}_t(x, y) = (y + \hat{V}_t^{\mathrm{RN}}(x))_+.$$

By Jensen's inequality, $B_t(x, y) \le V_t(x, y)$, but this may not hold for $\hat{B}_t$.
For $1 \le t < T$, since $B_t(x, y) \le V_t(x, y)$ and the maximum operator is nonexpansive, the one-step value-function error is bounded by

$$\left|\hat{V}_t(x, y) - V_t(x, y)\right| \le \max\left\{\beta_t^- + \max\{\epsilon_{t+1}^+, \epsilon_{t+1}^-\}, \rho_t^-\right\}. \tag{42}$$

At $t = 0$, the same bound follows directly from the nonexpansiveness of the minimum operator, since no risk-neutral maximum is applied.
Again, since the inequality above holds for an arbitrary state $(x_t, y_t) \in \mathcal{Z}_t^-$,

$$\epsilon_t^- \le \max\left\{\beta_t^- + \max\{\epsilon_{t+1}^+, \epsilon_{t+1}^-\}, \rho_t^-\right\}.$$

5.2.3 Combined error bound

Combining the one-step approximation error bounds derived for the risk-neutral and difficult regions, we have

$$\begin{aligned}\epsilon_t^+ &\le \beta_t^{\mathrm{RN}} + \epsilon_{t+1}^+,\\ \epsilon_t^- &\le \max\left\{\beta_t^- + \max\{\epsilon_{t+1}^+, \epsilon_{t+1}^-\}, \rho_t^-\right\}.\end{aligned}$$

Taking the maximum of the two inequalities gives

$$\begin{aligned}\max\{\epsilon_t^+, \epsilon_t^-\} &\le \max\left\{\beta_t^{\mathrm{RN}} + \epsilon_{t+1}^+, \beta_t^- + \max\{\epsilon_{t+1}^+, \epsilon_{t+1}^-\}, \rho_t^-\right\}\\ &\le \max\left\{\beta_t^{\mathrm{RN}} + \max\{\epsilon_{t+1}^+, \epsilon_{t+1}^-\}, \beta_t^- + \max\{\epsilon_{t+1}^+, \epsilon_{t+1}^-\}, \rho_t^-\right\}\\ &\le \max\{\beta_t^{\mathrm{RN}}, \beta_t^-, \rho_t^-\} + \max\{\epsilon_{t+1}^+, \epsilon_{t+1}^-\}.\end{aligned}$$

The final inequality follows from the nonnegativity of the error bounds. Since the terminal value function is exact, we have $\epsilon_T^+ = \epsilon_T^- = 0$. Applying the recursive inequality backward we obtain

$$\max\{\epsilon_t^+, \epsilon_t^-\} \le \sum_{s=t}^{T-1} \max\{\beta_s^{\mathrm{RN}}, \beta_s^-, \rho_s^-\}.$$

By the partition of the augmented-state domain,

$$\|\hat{V}_t - V_t\|_{\infty, \mathcal{Z}_t} = \max\{\epsilon_t^+, \epsilon_t^-\}.$$

Therefore, the total augmented-state approximation error bound is

$$\|\hat{V}_t - V_t\|_{\infty, \mathcal{Z}_t} \le \sum_{s=t}^{T-1} \max\{\beta_s^{\mathrm{RN}}, \beta_s^-, \rho_s^-\}. \tag{43}$$

5.3 Amplification by the AVaR confidence level

Since

$$J(\eta) = \eta + \frac{1}{1-\alpha} V_0(x_0, -\eta),$$

value-function approximation errors are amplified by

$$\frac{1}{1-\alpha}.$$

Thus, if

$$\left|\hat{V}_0(x_0, -\eta) - V_0(x_0, -\eta)\right| \le \epsilon,$$

then

$$\left|\hat{J}(\eta) - J(\eta)\right| \le \frac{\epsilon}{1-\alpha}. \tag{44}$$

The error ϵ can be bounded using the combined augmented-state approximation error derived in Section 5.2.3. The inequality shows that, for a fixed threshold η, the effect of value-function approximation error on the AVaR objective increases as α approaches one. Consequently, higher confidence levels require more accurate value-function approximations to maintain the same objective error bound.

5.4 Outer η discretization error

As discussed in Section 4, the outer optimization is performed over a finite set of thresholds $\mathcal{H} \subset \mathcal{I}_\eta$. Suppose that $\mathcal{H}$ is a grid containing the endpoints of $\mathcal{I}_\eta$, with maximum spacing h_η.
Let η^* be a global optimal threshold assumed to belong to a continuous interval $\mathcal{I}_\eta$, and let η^*_{grid} be the optimal threshold over the grid $\mathcal{H}$, evaluated using the exact objective J.
As stated by Bäuerle and Ott [2011] (Theorem 3.5), the value function is 1-Lipschitz continuous with respect to y. So,

$$|V_0(x_0, -\eta_1) - V_0(x_0, -\eta_2)| \leq |\eta_1 - \eta_2|. \tag{45}$$

Thus, the objective J is Lipschitz continuous, with a Lipschitz constant

$$\begin{aligned} |J(\eta_1) - J(\eta_2)| &= \left| \left(\eta_1 + \tfrac{1}{1-\alpha} V_0(x_0, -\eta_1)\right) - \left(\eta_2 + \tfrac{1}{1-\alpha} V_0(x_0, -\eta_2)\right) \right| \\ &= \left| (\eta_1 - \eta_2) + \tfrac{1}{1-\alpha}(V_0(x_0, -\eta_1) - V_0(x_0, -\eta_2)) \right| \\ &\leq \underbrace{\left(1 + \frac{1}{1-\alpha}\right)}_{L_\eta} |\eta_1 - \eta_2| \end{aligned} \tag{46}$$

The grid contains the endpoints of the interval $\mathcal{I}_\eta$, so there is a point $\eta' \in \mathcal{H}$ so that $|\eta' - \eta^*| \leq h_\eta/2$. Since η^*_{grid} is optimal in $\mathcal{H}$,

$$\begin{aligned} J(\eta^*_{\text{grid}}) - J(\eta^*) &\leq J(\eta') - J(\eta^*) \\ &\leq L_\eta \frac{h_\eta}{2}. \end{aligned} \tag{47}$$

However, in our algorithms we use the approximate objective $\hat{J}$; therefore, the selected threshold is not guaranteed to be the exact optimal η^*_{grid}. Let us define

$$\hat{\eta}_{\text{grid}} \in \arg\min_{\eta \in \mathcal{H}} \hat{J}(\eta).$$

Since $\hat{\eta}$ is optimal with respect to $\hat{J}$, we claim

$$\hat{J}(\hat{\eta}_{\text{grid}}) \leq \hat{J}(\eta^*_{\text{grid}}) \tag{48}$$

Let us define the uniform objective approximation error over the grid as δ. Then, using results from Section 5.3,

$$\begin{aligned} \delta &= \max_{\eta \in \mathcal{H}} |\hat{J}(\eta) - J(\eta)| \\ &= \max_{\eta \in \mathcal{H}} \left\{ \frac{1}{1-\alpha} \left| \hat{V}_0(x_0, -\eta) - V_0(x_0, -\eta) \right| \right\} \\ &\leq \frac{1}{1-\alpha} \|\hat{V}_0 - V_0\|_{\infty, \mathcal{Z}_0}. \end{aligned} \tag{49}$$

Using the optimality of $\hat{\eta}_{\text{grid}}$ with respect to $\hat{J}$, we have

$$\begin{aligned} J(\hat{\eta}_{\text{grid}}) - J(\eta^*_{\text{grid}}) &= \underbrace{\left[J(\hat{\eta}_{\text{grid}}) - \hat{J}(\hat{\eta}_{\text{grid}}) \right]}_{\leq \delta \text{ by (49)}} \\ &\quad + \underbrace{\left[\hat{J}(\hat{\eta}_{\text{grid}}) - \hat{J}(\eta^*_{\text{grid}}) \right]}_{\leq 0 \text{ by (48)}} \\ &\quad + \underbrace{\left[\hat{J}(\eta^*_{\text{grid}}) - J(\eta^*_{\text{grid}}) \right]}_{\leq \delta \text{ by (49)}} \\ &\leq 2\delta, \end{aligned} \tag{50}$$

Combining the discretization and approximation errors we obtain

$$\begin{aligned} J(\hat{\eta}_{\text{grid}}) - J(\eta^*) &= \underbrace{\left[J(\hat{\eta}_{\text{grid}}) - J(\eta^*_{\text{grid}})\right]}_{\leq 2\delta \text{ by (50)}} + \underbrace{\left[J(\eta^*_{\text{grid}}) - J(\eta^*)\right]}_{\leq L_\eta \frac{h_\eta}{2} \text{ by (47)}} \\ &\leq L_\eta \frac{h_\eta}{2} + 2\delta \\ &\leq L_\eta \frac{h_\eta}{2} + \frac{2}{1-\alpha} \|\hat{V}_0 - V_0\|_{\infty, \mathcal{Z}_0}. \end{aligned} \tag{51}$$

The bound separates the error introduced by the finite threshold grid from the error caused by approximating the objective function. The latter can be bounded using the combined augmented-state approximation error derived in Section 5.2.3.

5.5 Action-selection error and action gaps

The relationship between Q-function approximation errors and optimal action selection was studied by Farahmand [2011] through the action-gap phenomenon. The main observation is that correct action selection can be preserved despite approximation errors, provided that these errors are sufficiently small relative to the gap between optimal and suboptimal actions.
In Section 4.3.3 we introduced the predicted action gap $\Delta_t(s)$, calculated from the GP predictive means, as a component of the decision-uncertainty acquisition score. We now consider the corresponding gap defined using the exact Q-functions.
For an augmented state $s = (x, y)$, let the set of optimal actions be

$$\mathcal{A}^*_t(s) = \arg\min_{a \in \mathcal{A}} Q_t(s, a). \tag{52}$$

In the following analysis, we assume that at least one suboptimal action exists, so that the set $\mathcal{A} \setminus \mathcal{A}^*_t(s)$ is nonempty. Under this assumption, we define the true action gap as

$$\Delta^{\text{true}}_t(s) = \min_{a \in \mathcal{A} \setminus \mathcal{A}^*_t(s)} \left[Q_t(s, a) - V_t(s)\right]. \tag{53}$$

Let the local Q-function approximation error be

$$e^Q_t(s) = \max_{a \in \mathcal{A}} |\hat{Q}_t(s, a) - Q_t(s, a)|. \tag{54}$$

Recall that for $y \geq 0$, the approximate Q-function is $\hat{Q}_t(s, a) = y + \hat{Q}^{\text{RN}}_t(x, a)$.
The approximate greedy action is given by

$$\hat{a}_t(s) \in \arg\min_{a \in \mathcal{A}} \hat{Q}_t(s, a).$$

Proposition. If

$$\Delta^{\text{true}}_t(s) > 2e^Q_t(s), \tag{55}$$

then the approximate greedy action is optimal, i.e., $\hat{a}_t(s) \in \mathcal{A}^*_t(s)$.
Proof. Let $a^*_t(s) \in \mathcal{A}^*_t(s)$. By the optimality of $\hat{a}_t(s)$ with respect to $\hat{Q}_t$, we have

$$\begin{aligned} 0 \leq Q_t(s, \hat{a}_t(s)) - V_t(s) &\leq \left[Q_t(s, \hat{a}_t(s)) - \hat{Q}_t(s, \hat{a}_t(s))\right] \\ &\quad + \left[\hat{Q}_t(s, a^*_t(s)) - Q_t(s, a^*_t(s))\right] \\ &\leq 2e^Q_t(s). \end{aligned} \tag{56}$$

If $\hat{a}_t(s)$ were suboptimal, the definition of the true action gap would imply

$$Q_t(s, \hat{a}_t(s)) - V_t(s) \geq \Delta_t^{\text{true}}(s),$$

contradicting $\Delta_t^{\text{true}}(s) > 2e_t^Q(s)$. Therefore, $\hat{a}_t(s) \in \mathcal{A}_t^*(s)$. □

If all actions are optimal, any approximate greedy action is also optimal, and the conclusion holds trivially.

Equation (56) bounds the one-step action-selection error. The proposition further shows when the approximate greedy action is optimal. These results motivate decision-aware sampling, but do not guarantee the effectiveness of our acquisition score.

5.6 Policy-performance error

Let us define the uniform Q-function approximation error at time t as

$$e_t^Q = \sup_{s \in \mathcal{Z}_t} e_t^Q(s) \tag{57}$$

where $e_t^Q(s)$ is defined in (54).

For a fixed threshold η, let $\hat{\pi}_\eta$ denote the policy greedy with respect to $\hat{Q}_t$.

Proposition. The performance loss of $\hat{\pi}_\eta$ satisfies

$$0 \leq \mathbb{E}^{\hat{\pi}_\eta}\left[\left(C^{\hat{\pi}_\eta} - \eta\right)_+\right] - V_0(x_0, -\eta) \leq 2\sum_{t=0}^{T-1} e_t^Q. \tag{58}$$

Proof. By (56), the one-step action-selection error satisfies

$$0 \leq Q_t(s, \hat{\pi}_{\eta,t}(s)) - V_t(s) \leq 2e_t^Q.$$

By definition of Q_t and the law of total expectation, the one-step action selection error can be written as

$$\begin{aligned} Q_t(s, \hat{\pi}_{\eta,t}(s)) - V_t(s) &= \mathbb{E}^{\hat{\pi}_\eta}\left[V_{t+1}(s_{t+1}) - V_t(s) \mid s_t = s\right] \\ \mathbb{E}^{\hat{\pi}_\eta}\left[Q_t(s_t, \hat{\pi}_{\eta,t}(s_t)) - V_t(s_t)\right] &= \mathbb{E}^{\hat{\pi}_\eta}\left[\mathbb{E}^{\hat{\pi}_\eta}\left[V_{t+1}(s_{t+1}) - V_t(s_t) \mid s_t\right]\right] \\ &= \mathbb{E}^{\hat{\pi}_\eta}\left[V_{t+1}(s_{t+1}) - V_t(s_t)\right] \end{aligned}$$

Thus, summing these errors along the trajectory $t = 0, \ldots, T-1$:

$$\begin{aligned} \sum_{t=0}^{T-1} \mathbb{E}^{\hat{\pi}_\eta}\left[Q_t(s_t, \hat{\pi}_{\eta,t}(s_t)) - V_t(s_t)\right] &= \sum_{t=0}^{T-1} \mathbb{E}^{\hat{\pi}_\eta}\left[V_{t+1}(s_{t+1}) - V_t(s_t)\right] \\ &= \mathbb{E}^{\hat{\pi}_\eta}\left[V_T(s_T)\right] - V_0(x_0, -\eta) \\ &= \mathbb{E}^{\hat{\pi}_\eta}\left[\left(C^{\hat{\pi}_\eta} - \eta\right)_+\right] - V_0(x_0, -\eta). \end{aligned} \tag{59}$$

On the other hand, their summation can be bounded using (56):

$$\sum_{t=0}^{T-1} \mathbb{E}^{\hat{\pi}_\eta}\left[Q_t(s, \hat{\pi}_{\eta,t}(s)) - V_t(s)\right] \leq 2\sum_{t=0}^{T-1} e_t^Q. \tag{60}$$

Combining (59) and (60),

$$\mathbb{E}^{\hat{\pi}_\eta}\left[\left(C^{\hat{\pi}_\eta} - \eta\right)_+\right] - V_0(x_0, -\eta) \leq 2\sum_{t=0}^{T-1} e_t^Q.$$

The nonnegativity of the performance loss follows from the nonnegativity of each summed selection error. □

Corollary. Let $\hat{\eta}$ be the threshold selected by the approximate outer optimization, and let $\hat{\pi} = \hat{\pi}_{\hat{\eta}}$. Then,

$$\mathrm{AVaR}_\alpha\left(C^{\hat{\pi}}\right) - \min_\pi \mathrm{AVaR}_\alpha\left(C^\pi\right) \le L_\eta \frac{h_\eta}{2} + \frac{2}{1-\alpha}\|\hat{V}_0 - V_0\|_{\infty,\mathcal{Z}_0} + \frac{2}{1-\alpha}\sum_{t=0}^{T-1} e_t^Q. \tag{61}$$

Proof. By the Rockafellar–Uryasev representation,

$$\begin{aligned}\mathrm{AVaR}_\alpha\left(C^{\hat{\pi}}\right) &= \min_{\eta\in\mathbb{R}}\left\{\eta + \frac{1}{1-\alpha}\mathbb{E}^{\hat{\pi}}\left[\left(C^{\hat{\pi}} - \eta\right)_+\right]\right\} \\ &\le \hat{\eta} + \frac{1}{1-\alpha}\mathbb{E}^{\hat{\pi}}\left[\left(C^{\hat{\pi}} - \hat{\eta}\right)_+\right]\end{aligned}$$

Also, since η^* is a global optimizer,

$$\min_\pi \mathrm{AVaR}_\alpha\left(C^\pi\right) = J(\eta^*)$$

Thus, the difference can be written as

$$\begin{aligned}\mathrm{AVaR}_\alpha\left(C^{\hat{\pi}}\right) - \min_\pi \mathrm{AVaR}_\alpha\left(C^\pi\right) &\le \hat{\eta} + \frac{1}{1-\alpha}\mathbb{E}^{\hat{\pi}}\left[\left(C^{\hat{\pi}} - \hat{\eta}\right)_+\right] - J(\eta^*) \\ &= \hat{\eta} + \frac{1}{1-\alpha}\mathbb{E}^{\hat{\pi}}\left[\left(C^{\hat{\pi}} - \hat{\eta}\right)_+\right] - J(\eta^*) + J(\hat{\eta}) - J(\hat{\eta}) \\ &= J(\hat{\eta}) - J(\eta^*) \\ &\quad + \hat{\eta} + \frac{1}{1-\alpha}\mathbb{E}^{\hat{\pi}}\left[\left(C^{\hat{\pi}} - \hat{\eta}\right)_+\right] - J(\hat{\eta}) \\ &= J(\hat{\eta}) - J(\eta^*) \\ &\quad + \frac{1}{1-\alpha}\left(\mathbb{E}^{\hat{\pi}}\left[\left(C^{\hat{\pi}} - \hat{\eta}\right)_+\right] - V_0(x_0, -\hat{\eta})\right).\end{aligned}$$

Applying (51) and the preceding proposition gives the result. □

6 Experimental setup

We consider the stochastic control problem

$$x_{t+1} = x_t + a_t + \xi_t, \quad c_t(x_t, a_t) = x_t^2 + a_t^2, \quad c_T(x_T) = x_T^2, \tag{62}$$

with $\mathcal{A} = \{-1, 0, 1\}$, $x_0 = 1$, horizons $T = 3, 7$, and confidence level $\alpha = 0.95$. We minimize the static AVaR of cumulative cost. The disturbances follow a discrete approximation of the standard normal distribution with 201 equally spaced points on $[-5, 5]$. Their probabilities are proportional to the normal density and normalized to sum to one. All methods use this distribution for Bellman expectations and policy evaluation.

6.1 Reference solutions and thresholds

For $T = 3$, we compute the exact DP solution for the discrete disturbance distribution and search over

$$\mathcal{H} = \{0, 0.25, 0.50, \ldots, 40\}.$$

The grid-optimal threshold is $\eta = 13.75$. The reference policy is evaluated by enumerating all disturbance paths.

For $T = 7$, we estimate the threshold using a pilot baseline ADP. We first estimate the VaR of the fitted risk-neutral policy, train a pilot ADP at this threshold, and use its empirical VaR, $\eta = 27.8325$. The pilot and later benchmarks share an evaluation seed, so threshold selection is not fully independent of evaluation.

All approximate methods use the same fitted risk-neutral continuation for each horizon. The decomposition for $y \geq 0$ is exact, but its continuation values are approximated as described in Section 3.5.

6.2 Experimental comparisons

We conduct four experiments. First, we validate baseline ADP against exact DP at $T = 3$, comparing root values, policy performance, and action agreement across backup budgets. Unseen states are initialized using 20 Monte Carlo rollouts.

Second, we compare six probabilistic ADP sampling rules: random, uncertainty, decision-aware, and decision-aware with reachability, tail relevance, or both. The four decision-based rules use

$$D_t(s), \quad D_t(s)R_t(s), \quad D_t(s)\tilde{L}_t(s), \quad D_t(s)R_t(s)\tilde{L}_t(s), \tag{63}$$

where $\tilde{L}_t(s) = 0.5 + 0.5L_t(s)$ prevents zero tail weights.

At each stage $t = 1, \ldots, T-1$, we generate 5,000 uniform candidates from $[-8, 8] \times [-\eta, 0)$, shared across methods and budgets for each seed. Active acquisition starts with 20 random states and adds batches of 10. Each selected state receives backups for all three actions. Random sampling fits its GPs once per stage.

We use RBF-kernel GPs with normalized-input length scales $(0.16, 0.16)$, regularization 10^{-6}, and standardized targets. Reachability and tail scores use 2,000 risk-neutral policy rollouts and $k = 50$ nearest neighbors. Saved baseline policies trained for 1,000 episodes serve as references: three for $T = 3$ and one for $T = 7$.

Third, we study outer-threshold optimization using random, uncertainty, and decision-aware sampling. One collection of GPs is trained on $[-8, 8] \times [-40, 0)$ and reused to evaluate all thresholds in $\mathcal{H}$. We select the threshold minimizing the predicted RU objective and evaluate its policy. One saved baseline policy per horizon serves as a reference.

Finally, we compare candidate pools of 1,000 to 5,000 states per stage, in increments of 1,000, for random and decision-aware sampling. Smaller pools are subsets of the same 5,000-point pool. Other settings remain fixed.

6.3 Budgets and repeated runs

One Bellman backup evaluates the Bellman expectation for one state-action pair. We use nominal budgets

$$\mathcal{B} = \{900, 1800, 2700, 3600, 5400, 7200, 9000\}. \tag{64}$$

The fixed-threshold experiment at $T = 3$ uses only the first four budgets.

For probabilistic ADP, the remaining budget is divided equally among stages $1, \ldots, T-1$ after reserving root backups: three for fixed η and $3|\mathcal{H}| = 483$ for outer optimization. Actual counts may be slightly smaller due to action grouping. Budgets exclude GP fitting, acquisition, initialization, risk-neutral preparation, and evaluation. We report actual backups and training and evaluation times separately.

Table 1 summarizes the runs. The three-seed experiments use seeds 47-49, and the ten-seed experiments use 47-56.

Table 1: Training seeds and evaluation trajectories.

Experiment	Horizon	Seeds	Trajectories
Baseline validation	3	3	2,000
Fixed threshold	3	10	2,000
Fixed threshold	7	3	2,000
Outer threshold	3 and 7	10 each	2,000
Candidate pool	3 and 7	3 each	20,000

6.4 Policy evaluation

We evaluate policies using cumulative costs from shared disturbance trajectories to reduce Monte Carlo comparison noise. For costs $C^{(1)}, \ldots, C^{(N)}$, we report mean cost and the empirical RU objective

$$\hat{J}_{\text{eval}}(\eta) = \eta + \frac{1}{N(1-\alpha)} \sum_{i=1}^{N} (C^{(i)} - \eta)_+. \tag{65}$$

Empirical AVaR minimizes this expression over η. It differs from the evaluated RU at the selected threshold and from the predicted objective

$$\hat{J}(\eta) = \eta + \hat{V}_0(x_0, -\eta)/(1-\alpha).$$

For $T = 3$, we also measure disagreement with the exact policy on 101 test entries: the initial state and 50 states sampled with replacement at each of $t = 1, 2$, according to exact-policy visitation probabilities.

We report means and standard deviations across training seeds, separately from Monte Carlo uncertainty. In paired evaluations, empirical AVaR is computed from the full set of trajectories, while the RU objective allows comparisons trajectory by trajectory. The candidate-pool experiment uses more evaluation trajectories, so its AVaR estimates have different precision and should not be compared directly with those from other experiments.

Baseline ADP may randomly initialize and store values for unseen states during evaluation. Its results can therefore depend on the order of evaluation. For this reason, one should treat baseline comparisons as descriptive rather than conclusive.

7 Results & discussion

7.1 Baseline validation against exact dynamic programming

We compare baseline ADP with exact DP for $T = 3$ and $\eta = 13.75$. The exact fixed-threshold RU objective is 18.4911, while the corresponding policy AVaR is 18.4900, since η is optimal only on the chosen grid.

Figure 1 shows the estimated root RU objective against Bellman backups. At the largest budget, the mean estimate is 18.8607 with standard deviation 0.0298, giving an error of 0.3696. The estimates become more stable across seeds but remain above the exact value, without monotone error reduction.

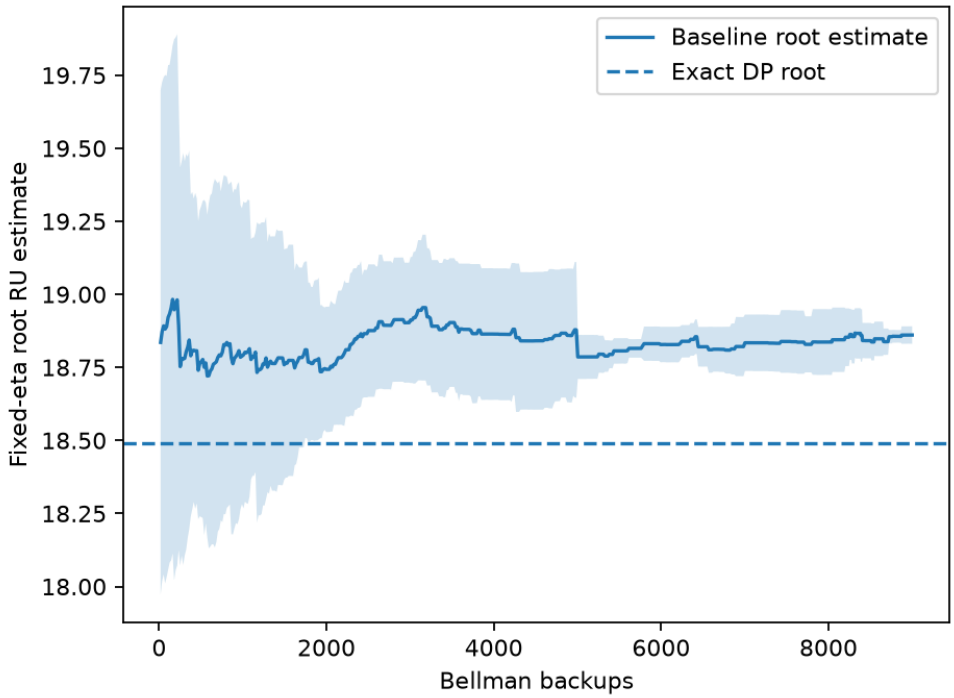


Figure 1: Root RU estimates with training-seed standard deviations.

We evaluate policy quality on 2,000 shared disturbance trajectories. Table 2 reports paired differences averaged over three training seeds. The root RU error is the mean absolute prediction error. The RU and AVaR gaps compare evaluated policies using the fixed threshold and empirical AVaR, respectively. Positive gaps favor exact DP.

Table 2: Baseline errors relative to exact DP.

Backups	Root RU error	RU gap	AVaR gap	Mean-cost gap
900	0.4934	0.0065	0.0055	0.0114
1,800	0.2965	0.0191	0.0166	0.0074
2,700	0.4030	0.0145	0.0111	0.0035
3,600	0.3903	0.0201	0.0166	0.0111
5,400	0.3035	0.0065	0.0055	0.0114
7,200	0.3429	0.0065	0.0055	0.0114
8,998	0.3696	0.0065	0.0055	0.0114

The backup counts are averaged across seeds. Policy gaps remain small at all budgets, with no clear improvement as the budget increases. In contrast, root-value errors are noticeably larger.
Paired evaluation is important: the exact policy has an empirical AVaR of 19.6384 on the shared sample, compared with 18.4900 under full path enumeration. Thus, direct comparison with the enumerated value would mix policy and Monte Carlo errors.
Figure 2 shows the paired policy gaps. All 95% Monte Carlo intervals for RU differences include zero. However, baseline evaluation may depend on the order in which states are visited because of random initialization and caching. Therefore, these results should be interpreted with caution and do not establish equal policy performance.

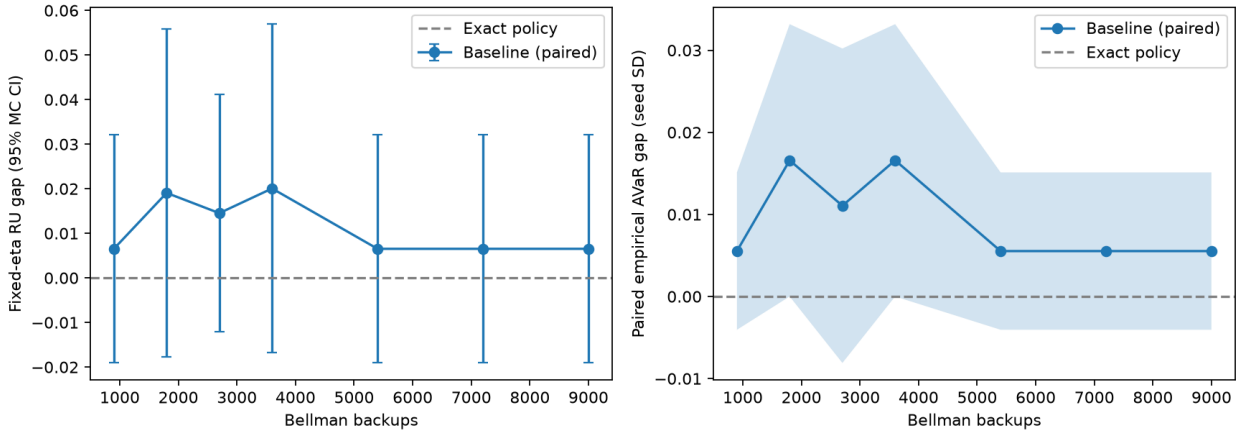


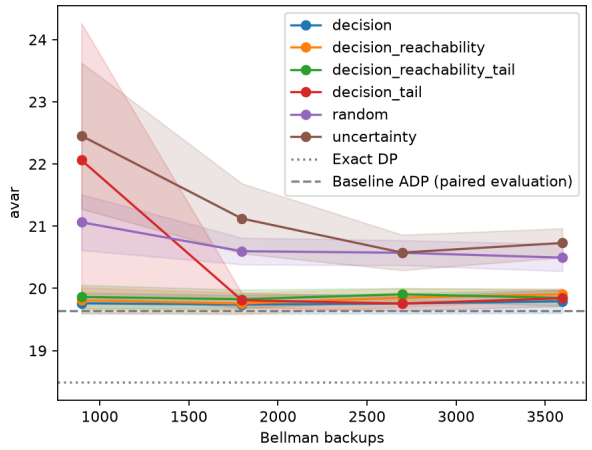
Figure 2: Paired policy gaps. Left: nominal 95% Monte Carlo intervals. Right: training-seed standard deviations.

The baseline also agrees with the exact policy at all 101 sampled test entries, for every seed and budget.
Overall, baseline ADP achieves policy performance close to exact DP despite noticeable root-value errors.

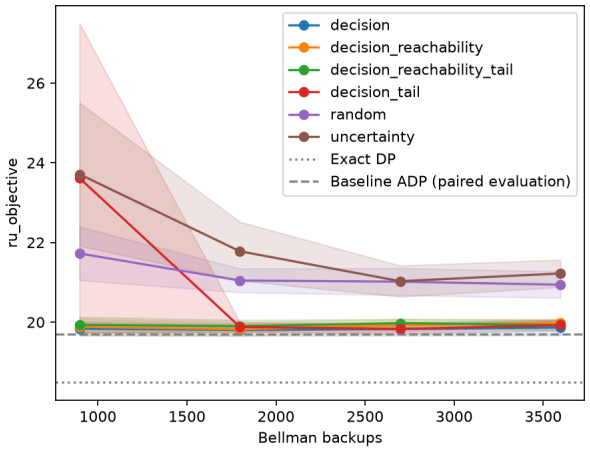
7.2 Fixed-threshold probabilistic ADP

We compare six probabilistic ADP sampling rules at $\eta = 13.75$ for $T = 3$ and $\eta = 27.8325$ for $T = 7$, evaluating policy quality across Bellman-backup budgets.

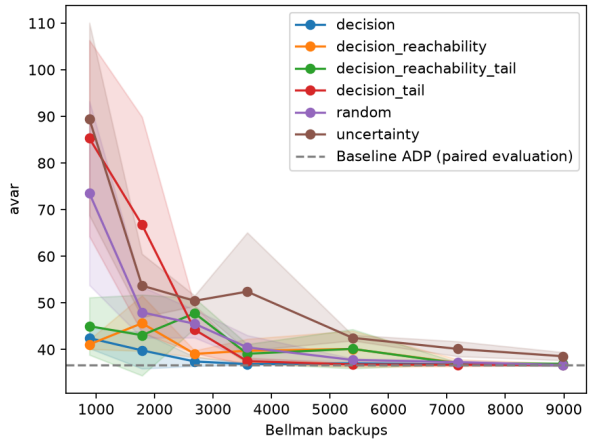
Empirical AVaR and RU objective. Figure 3 shows empirical AVaR and evaluated fixed-threshold RU objectives, both calculated from policy rollouts. For $T = 3$, decision-aware sampling gives lower AVaR than random and uncertainty sampling at every budget. At 3,597 backups, the mean values are 19.7929, 20.4940, and 20.7308, respectively. Decision-aware performance is already good at the smallest budget and changes little afterward.
For $T = 7$, decision-aware AVaR decreases from 42.4413 at 885 backups to 36.7091 at 8,985 backups. At 3,585 backups, it achieves 36.8594, compared with 40.4588 for random and 52.4161 for uncertainty sampling.



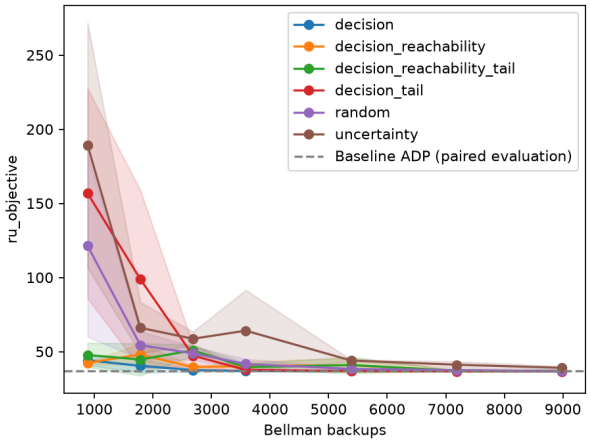
(a) Empirical AVaR, $T = 3$.



(b) Evaluated RU, $T = 3$.



(c) Empirical AVaR, $T = 7$.



(d) Evaluated RU, $T = 7$.

Figure 3: Empirical AVaR and evaluated RU objective against Bellman backups. Shading shows training-seed standard deviations.

Comparison with exact DP. For $T = 3$, the exact policy has AVaR 18.4900 under full enumeration, but 19.6384 on the shared 2,000-trajectory evaluation sample. Thus, comparing GP results directly with the enumerated reference includes significant Monte Carlo error.

Figure 4 shows paired RU gaps against exact DP. At 3,597 backups, decision-aware sampling has a mean gap of 0.1704, with approximate 95% Monte Carlo interval $[-0.0137, 0.3545]$. Random and uncertainty sampling give larger gaps of 1.2430 and 1.5251, with intervals above zero. All decision-aware intervals include zero, but this does not establish equality with the exact policy. The intervals measure Monte Carlo uncertainty after averaging across training seeds.

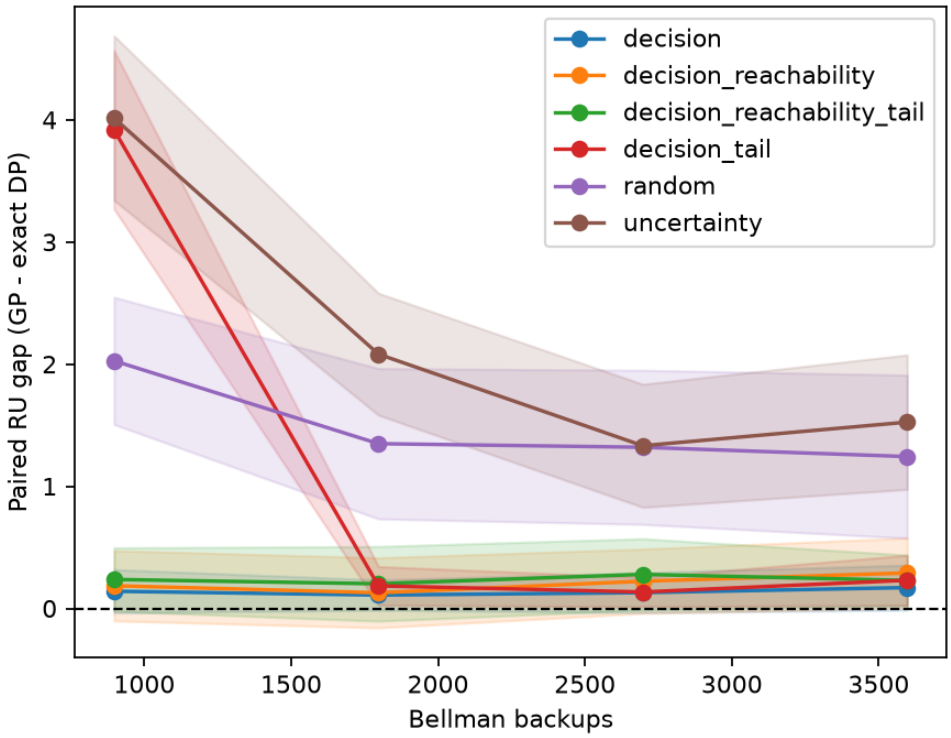


Figure 4: Paired RU gaps against exact DP at $T = 3$, with approximate 95% Monte Carlo intervals.

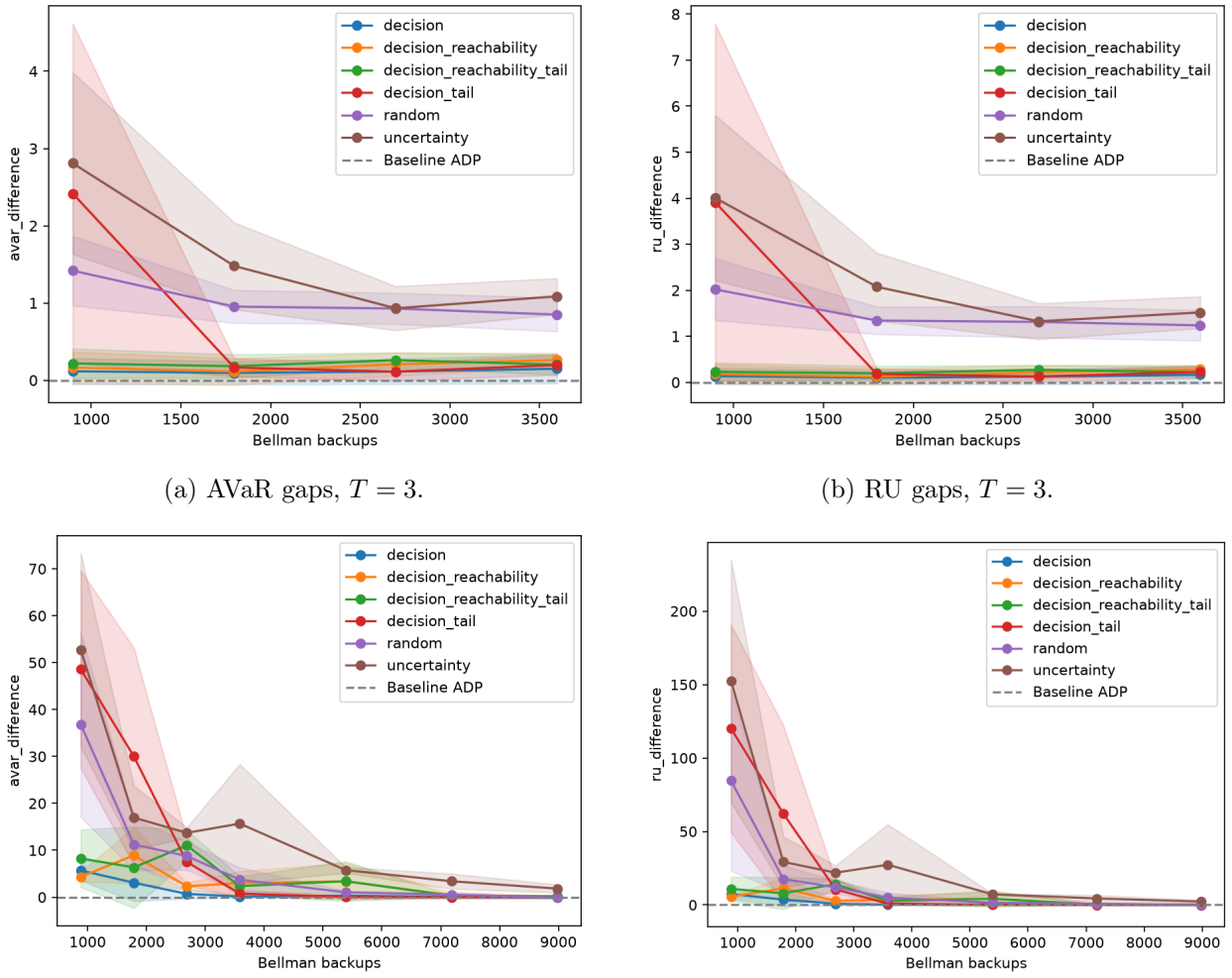
Comparison with baseline ADP. Table 3 reports paired differences against saved baseline policies at the largest budgets. Negative gaps favor probabilistic ADP. The reference uses three baseline policies for $T = 3$ and one for $T = 7$.

Table 3: Paired gaps relative to baseline ADP.

	$T = 3$: 3,597 backups		$T = 7$: 8,985 backups	
Method	AVaR gap	RU gap	AVaR gap	RU gap
Random	0.8500	1.2365	-0.0636	0.1058
Uncertainty	1.0869	1.5186	1.8196	2.3121
Decision	0.1489	0.1639	-0.0255	-0.0203
Decision + reachability	0.2625	0.2837	0.0558	0.0553
Decision + tail	0.1972	0.2242	-0.0086	-0.0082
Decision + reachability + tail	0.2034	0.2223	0.2043	0.2037

For $T = 7$, decision-aware sampling approaches baseline performance by 3,585 backups. At the largest budget, its gaps are slightly negative but small relative to training-seed variation (standard deviations 0.2956 for AVaR and 0.3051 for RU). Random sampling gives a slightly lower empirical AVaR at this budget, but a higher RU objective and mean cost.

Figure 5 shows these comparisons across budgets. They remain descriptive because baseline evaluation can depend on randomized initialization and caching. The $T = 7$ baseline is not an exact reference.



(a) AVaR gaps, $T = 3$.

(b) RU gaps, $T = 3$.

(c) AVaR gaps, $T = 7$.

(d) RU gaps, $T = 7$.

Figure 5: Paired AVaR and RU gaps to baseline ADP. Shading shows training-seed standard deviations.

Acquisition rules and computational cost. Adding reachability or tail relevance does not consistently improve decision-aware sampling. Tail weighting is particularly unstable at small budgets, giving mean AVaR 85.2944 for $T = 7$ at 885 backups.

At $T = 3$ and 3,597 backups, decision-aware sampling disagrees with the exact policy on only 4.46% of sampled test entries, compared with 38.61% for random and 40.10% for uncertainty sampling.

Predicted objectives also differ from evaluated risk. For decision-aware sampling at $T = 7$ and 8,985 backups, predicted and evaluated RU objectives are 38.6927 and 36.7169, respectively.

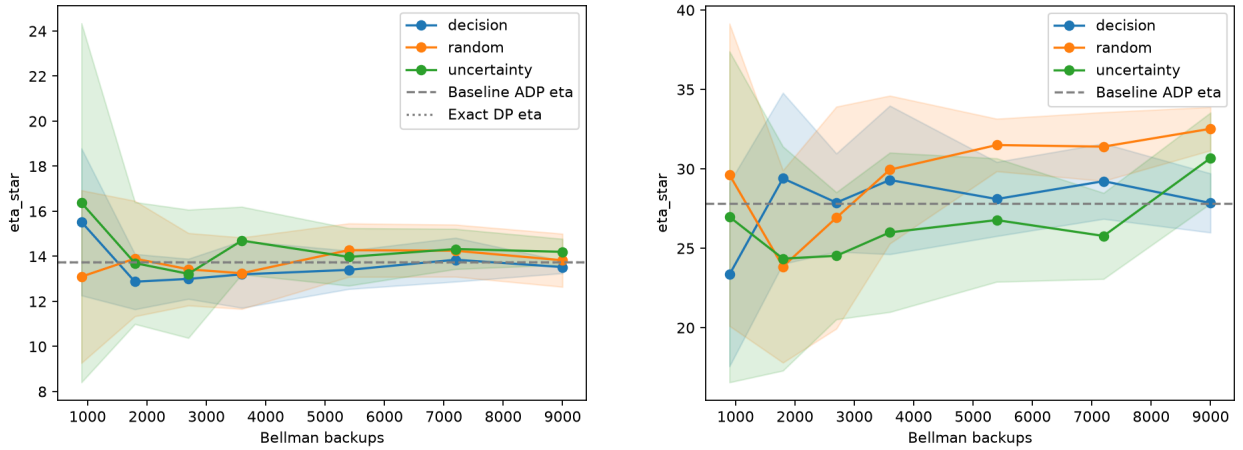
The saved baselines use approximately 18,000 training backups for $T = 3$ and 41,766 for $T = 7$. Decision-aware GP sampling achieves comparable policy performance with substantially fewer backups and lower recorded training times in these runs. However, this does not establish general speedup factors or the minimum baseline budget needed.

Within probabilistic ADP, active sampling has additional computational costs. At 8,985 backups and $T = 7$, decision-aware training takes about 243 seconds, compared with 60 seconds for random sampling. Thus, backup efficiency does not necessarily imply runtime efficiency.

7.3 Outer-threshold optimization

We compare random, uncertainty, and decision-aware sampling over $\mathcal{H} = \{0, 0.25, \ldots, 40\}$, using one collection of GP models for all thresholds. The threshold minimizing the predicted RU objective is selected, and its policy is evaluated over shared trajectories.
The outer search requires 483 root backups. At the smallest budget of 897 backups, only 414 remain for GP training.

Selected thresholds. Figure 6 shows the selected thresholds. At 8,997 backups, decision-aware sampling gives mean $\hat{\eta} = 13.525$ for $T = 3$, close to the exact grid-optimal value 13.75. For $T = 7$, it gives $\hat{\eta} = 27.850$, close to the heuristic pilot value 27.8325. Neither agreement establishes optimality beyond the available reference.
One random-sampling run at the smallest $T = 7$ budget selects the upper boundary $\eta = 40$, indicating a possible limitation of the search interval.

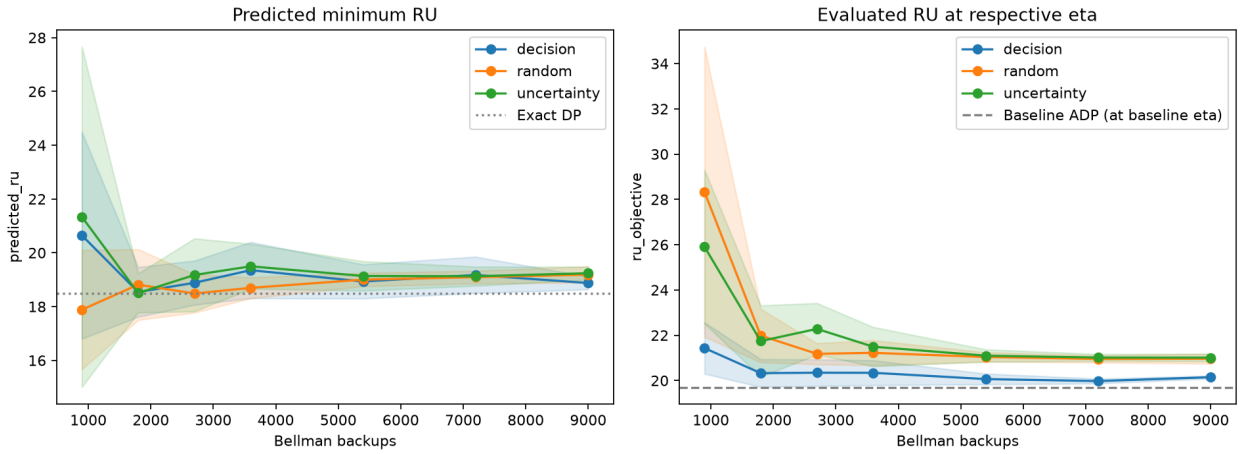


(a) Selected thresholds, $T = 3$.

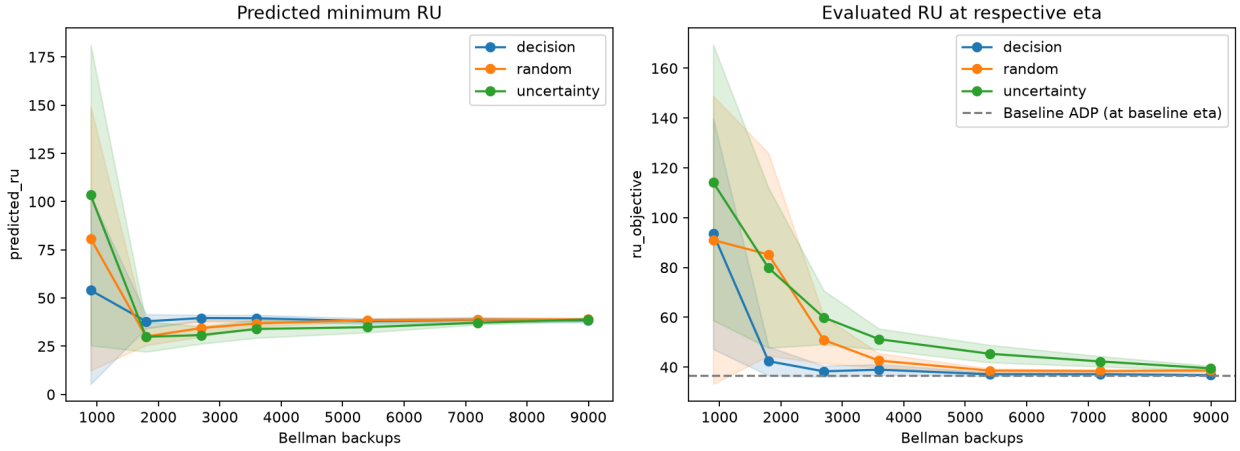
(b) Selected thresholds, $T = 7$.

Figure 6: Selected thresholds against Bellman backups. Shading shows training-seed standard deviations.

Predicted and evaluated objectives. Figure 7 compares predicted minimum RU objectives with those evaluated from policy rollouts. After GP training, predictions require only root Bellman evaluations, without Monte Carlo policy rollouts. Their cost is independent of the evaluation sample size.
However, predictions are not always accurate. For $T = 3$ at 897 backups, random sampling predicts 17.8852, below the exact grid minimum 18.4911, while its evaluated RU is 28.3359. At the largest budget, decision-aware sampling gives predicted and evaluated RU values of 18.8761 and 20.1483 for $T = 3$, and 38.3366 and 36.7794 for $T = 7$. These differences reflect both approximation and Monte Carlo evaluation errors, which are not separated in this experiment.



(a) Predicted minimum and evaluated RU, $T = 3$.



(b) Predicted minimum and evaluated RU, $T = 7$.

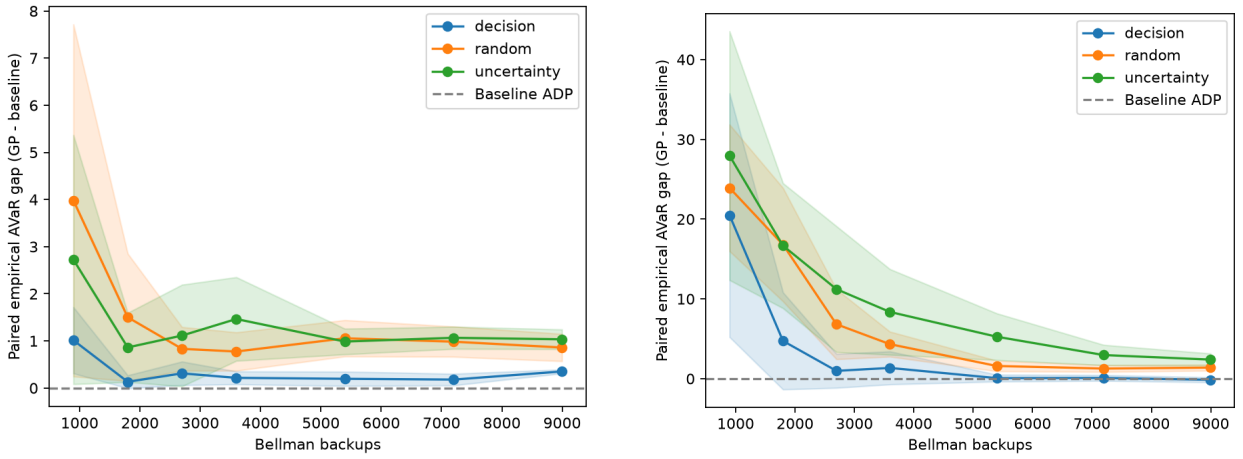
Figure 7: Predicted minimum RU (left) and evaluated RU (right) against Bellman backups. Shading shows training-seed standard deviations.

Evaluated policy performance. Table 4 reports the results at the largest budget. Decision-aware sampling achieves the lowest mean empirical AVaR and mean cost for both horizons. For $T = 7$, its AVaR decreases from 57.2183 at 897 backups to 36.5893 at 8,997 backups. The improvement is less consistent for $T = 3$.

Table 4: Outer-threshold results at 8,997 backups. Means across ten training seeds.

T	Method	$\hat{\eta}$	AVaR	Evaluated RU	Mean cost
3	Random	13.825	20.4978	20.9621	7.4354
	Uncertainty	14.200	20.6727	21.0112	7.5685
	Decision	13.525	19.9907	20.1483	6.6625
7	Random	32.525	38.1217	38.6228	16.1180
	Uncertainty	30.675	39.1253	39.5088	16.7823
	Decision	27.850	36.5893	36.7794	14.7408

Paired evaluation. Figure 8 compares empirical AVaR against one saved fixed-threshold baseline policy per horizon, using shared disturbance trajectories.
At the largest budget, decision-aware sampling gives mean paired AVaR gaps of 0.3523 for $T = 3$ and -0.1454 for $T = 7$. The latter has training-seed standard deviation 0.3136, so it does not establish superiority over the baseline. Random and uncertainty sampling give larger gaps of 1.3871 and 2.3906 for $T = 7$.



(a) Paired AVaR gaps, $T = 3$.

(b) Paired AVaR gaps, $T = 7$.

Figure 8: Paired empirical AVaR gaps to baseline ADP against Bellman backups. Shading shows training-seed standard deviations.

We also compare RU gaps using the baseline threshold and each policy's own threshold, without changing the policies. For decision-aware sampling at $T = 7$ and 8,997 backups, the gaps are -0.1437 and 0.0421, with nominal 95% Monte Carlo intervals $[-0.8676, 0.5803]$ and $[-0.6857, 0.7700]$, respectively. Both differences are small relative to their uncertainty, but baseline evaluation may depend on evaluation order. These results also show that the selected threshold does not always minimize the empirical RU objective.

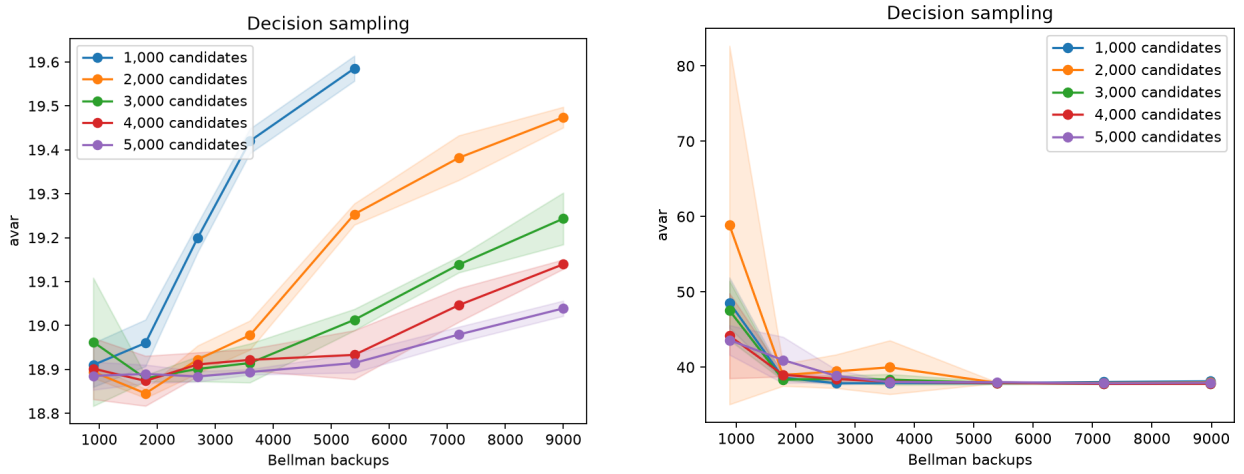
Interpretation and computational cost. Decision-aware sampling gives better threshold and policy estimates than the other sampling methods in these experiments. However, we cannot conclude that outer optimization is better than fixed-threshold training, since the experiments use different candidate domains, budgets, and numbers of seeds.
Decision-aware sampling is also slower than random sampling. At the largest budget, mean training times are about 2,219 versus 98 seconds for $T = 3$, and 412 versus 100 seconds for $T = 7$. The higher runtime for $T = 3$ may be explained by the larger number of training points per GP, since the backup budget is distributed across fewer stages. As GP fitting has roughly cubic complexity, training can be slower for shorter horizons despite having fewer stages.

7.4 Candidate-pool ablation

We compare random and decision-aware sampling with 1,000 to 5,000 candidates per stage. Smaller pools are subsets of the same 5,000-point pool, with other training settings fixed. We use three training seeds and 20,000 shared evaluation trajectories. Due to different evaluation sample sizes, raw AVaR estimates are not directly compared with earlier experiments.

Effect on decision-aware sampling. Figure 9 shows empirical AVaR across budgets. For $T = 3$, larger pools generally improve performance. At 3,597 backups, increasing the pool from 1,000 to 5,000 candidates reduces mean AVaR from 19.4200 to 18.8939 and sampled policy disagreement from 32.01% to 4.62%.

For $T = 7$, the differences between pool sizes become small as the budget increases. At 8,985 backups, all methods give similar AVaR values, with no clear advantage from larger candidate pools. This suggests that pool size has little effect on policy performance at larger budgets.



(a) Decision-aware AVaR, $T = 3$.

(b) Decision-aware AVaR, $T = 7$.

Figure 9: Decision-aware AVaR for different pool sizes. Shading shows training-seed standard deviations.

Table 5 summarizes the results at the nominal budget of 3,600 backups.

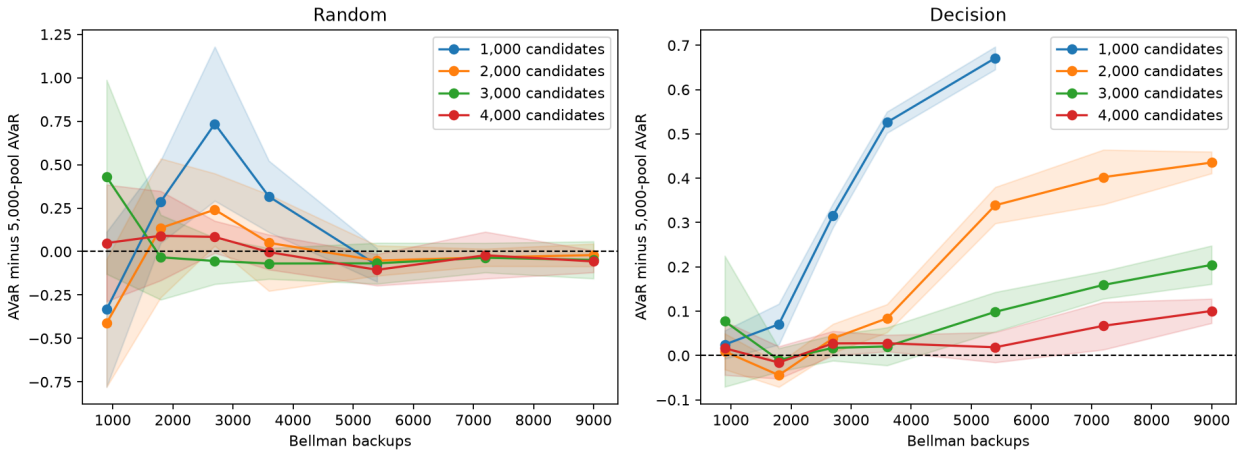
Table 5: Decision-aware sampling at 3,600 nominal backups.

Candidates per stage	$T = 3$ AVaR	$T = 3$ disagreement (%)	$T = 7$ AVaR
1,000	19.4200	32.01	37.8834
2,000	18.9777	13.20	39.9740
3,000	18.9143	8.25	38.3482
4,000	18.9216	7.59	38.0603
5,000	18.8939	4.62	38.0193

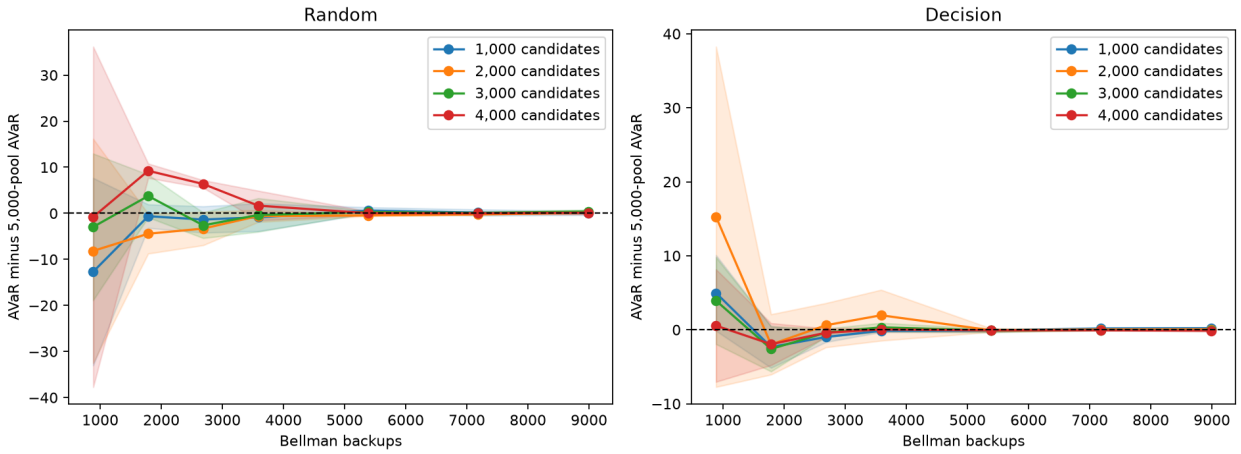
Paired comparisons between pool sizes. Figure 10 shows paired empirical AVaR differences relative to the 5,000-point pool, using the same seeds, budgets, and disturbance paths. Positive differences favor the larger pool.

For decision-aware sampling at $T = 3$, smaller pools become less effective at larger budgets. At 3,597 backups, the 1,000-point pool has a mean AVaR gap of 0.5261, with training-seed standard deviation 0.0239. Random sampling shows no similarly clear ordering, since it does not rank candidates by acquisition scores.

For $T = 7$, differences between pools become small at larger budgets. With only three training seeds, the experiment does not establish a reliable ranking of pool sizes.



(a) Paired AVaR gaps, $T = 3$.



(b) Paired AVaR gaps, $T = 7$.

Figure 10: Paired AVaR gaps relative to 5,000 candidates. Shading shows training-seed standard deviations.

Pool utilization and increasing budgets. More backups do not always improve the policy. For $T = 3$, decision-aware AVaR with 1,000 candidates increases from 18.9099 at 897 backups to 19.5854 at 5,397 backups. With 5,000 candidates, the increase is smaller, from 18.8852 to 18.9144. At 5,397 backups, the algorithm uses 899 states per stage, corresponding to 89.9% of the smaller pool but only 17.98% of the larger one. High pool utilization leaves less freedom for acquisition, which may contribute to the deterioration.

However, the experiment does not establish its cause. GP refitting and backward propagation of approximate Bellman targets may also contribute. Even with 5,000 candidates, AVaR rises to 19.0388 at 8,997 backups. Thus, larger pools reduce but do not eliminate the observed deterioration. Using fewer backups or larger pools may help, but neither guarantees better performance.

Computational trade-off. Larger pools increase acquisition costs. At 3,600 nominal backups, increasing the pool from 1,000 to 5,000 candidates raises mean training time from 192 to 257 seconds for $T = 3$, and from 68 to 82 seconds for $T = 7$, without changing the number of backups.

Candidate-pool size is therefore an additional computational parameter that should be considered together with the Bellman-backup budget.

7.5 Discussion

The main finding is that decision-aware sampling often achieves better policy performance per Bellman backup than random or uncertainty sampling, especially at moderate budgets. Adding reachability or tail weighting does not consistently improve these results.

Value accuracy and policy quality. The baseline ADP achieves policy performance close to exact DP despite noticeable root-value errors. This shows that accurate value estimation and accurate action selection are different objectives. As shown by the action-gap result in Section 5.5, approximation errors may preserve the optimal action when they are sufficiently small relative to the true action gap. The experiments are consistent with this result but do not verify the condition at every state.
Outer-threshold optimization additionally depends on the accuracy of the predicted RU objective. A good fixed-threshold policy or a small predicted objective does not guarantee a good selected threshold. The experiments also do not establish that outer optimization improves on fixed-threshold training, since their configurations and budget allocations differ.

Paired evaluation. Paired evaluation reduces Monte Carlo noise when comparing policies. For $T = 3$, much of the difference from the exact AVaR comes from the finite evaluation sample. However, pairing does not remove actual policy differences or sampling uncertainty.
All training seeds use the same evaluation trajectories, so averaging over seeds does not remove sampling error. Also, empirical AVaR and fixed-threshold RU may rank policies differently because only AVaR minimizes over the threshold.

Reachability and tail weighting. The additional scores use trajectories generated by the fitted risk-neutral policy, which may not represent the states important under the AVaR policy. Multiplying these scores by decision uncertainty may also reduce the priority of useful states. Furthermore, threshold exceedance is only a proxy for the effect of action errors on AVaR.
These are possible explanations for the mixed results. Experiments on tail relevance weighting did not show meaningful improvements either. Thus, better ways of incorporating reachability and tail relevance might be a subject for further research.

Candidate pools and computational cost. The candidate-pool experiment shows that more Bellman backups do not always improve policy performance. Smaller pools can restrict acquisition when most candidates are already selected. Larger pools reduce the observed deterioration for $T = 3$, but do not guarantee improvement. GP refitting and backward propagation of approximation errors may also contribute, although their effects were not isolated.
Backup efficiency also differs from runtime efficiency. Repeated GP fitting and acquisition calculations can make decision-aware sampling slower than random sampling. However, decision-only acquisition avoids the additional rollouts and nearest-neighbor calculations required for reachability and tail scores. Its accuracy and simpler acquisition procedure make it attractive, although the experiments do not establish a consistent accuracy-runtime advantage over the weighted variants.

Further threshold-search improvements. The same trained GP models can evaluate the predicted RU objective at different thresholds. An adaptive search could therefore reduce root evaluations and leave more budget for GP training. However, the optimized RU objective is not necessarily convex or unimodal, so local optimization does not guarantee a global minimum.

Limitations. The experiments consider one scalar control problem, two horizons, and one confidence level. Exact DP is available only for $T = 3$ under discretized disturbances, and its threshold is grid-optimal. For $T = 7$, the fixed threshold is heuristic, and its pilot estimation shares an evaluation seed with later benchmarks.
Baseline evaluation may randomly initialize and cache unseen states. Evaluation-order dependence was confirmed for the saved $T = 7$ policy, but its effect on benchmark rankings was not established. Baseline comparisons and their Monte Carlo intervals therefore require caution.
GP uncertainty is model-based and does not guarantee small Bellman approximation errors. The risk-neutral continuation is also approximated, despite the exact mathematical decomposition for $y \geq 0$. Finally, some experiments use only three training seeds, and shared evaluation trajectories introduce common Monte Carlo uncertainty.
Thus, the results support decision-aware Bellman allocation within the tested settings, without establishing general superiority or convergence. The main remaining question is how reachability and tail information can improve policy accuracy beyond decision uncertainty alone.

8 Conclusion

In this paper, we studied finite-horizon static AVaR control using approximate dynamic programming. We proposed a probabilistic ADP method based on Gaussian processes, which approximates Q-functions in the difficult region of the augmented state space and allocates Bellman backups according to decision uncertainty, reachability, and tail relevance. We also derived approximation and policy-performance error bounds, accounting for the risk-neutral decomposition and outer threshold optimization.
The experiments show that decision-aware sampling often achieves better policy performance per Bellman backup than random or uncertainty sampling, especially at moderate computational budgets. However, adding reachability or tail weighting does not consistently improve performance. We also observed that better value-function estimates do not necessarily produce better policies, and that increasing the number of Bellman backups does not always improve the resulting AVaR. These findings suggest that allocating computational effort according to uncertainty about the optimal action is a promising approach to risk-averse ADP. Nevertheless, the experiments are limited to one control problem and do not establish general superiority or convergence. Future work may investigate more effective ways to incorporate reachability and tail relevance, as well as extensions to more complex control problems.

Code availability

The source code and experimental scripts used in this study are publicly available at `https://github.com/dilzhan/avar-mdp`.

References

Nicole Bäuerle and Jonathan Ott. Markov decision processes with average-value-at-risk criteria. *Mathematical Methods of Operations Research*, 74:361–379, December 2011. doi: 10.1007/s00186-011-0367-0.

Pauline Bourigault, Xiaotong Ji, Matthieu Zimmer, Rasul Tutunov, and Haitham Bou-Ammar. Tail-influence sampling for CVaR policy evaluation, 2026.

Sait Cakmak, Yuhao Wang, Siyang Gao, and Enlu Zhou. Contextual ranking and selection with Gaussian processes and optimal computing budget allocation. *ACM Transactions on Modeling and Computer Simulation*, 34(2), 2024. doi: 10.1145/3633456.

Yinlam Chow, Aviv Tamar, Shie Mannor, and Marco Pavone. Risk-sensitive and robust decision-making: a CVaR optimization approach. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

Yaakov Engel, Shie Mannor, and Ron Meir. Reinforcement learning with Gaussian processes. In *Proceedings of the 22nd International Conference on Machine Learning*, pages 201–208, 2005. doi: 10.1145/1102351.1102377.

Amir-massoud Farahmand. Action-gap phenomenon in reinforcement learning. In J. Shawe-Taylor, R. Zemel, P. Bartlett, F. Pereira, and K. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 24. Curran Associates, Inc., 2011. URL `https://proceedings.neurips.cc/paper_files/paper/2011/file/013d407166ec4fa56eb1e1f8cbe183b9-Paper.pdf`.

Robert Grande, Thomas Walsh, and Jonathan How. Sample efficient reinforcement learning with Gaussian processes. In *Proceedings of the 31st International Conference on Machine Learning*, volume 32 of *Proceedings of Machine Learning Research*, pages 1332–1340, 2014.

Daniel R. Jiang and Warren B. Powell. Risk-averse approximate dynamic programming with quantile-based risk measures. *Mathematics of Operations Research*, 2018. doi: 10.1287/moor.2017.0872.

Aneri Muni, Vincent Taboga, Esther Derman, Pierre-Luc Bacon, and Erick Delage. Reward redistribution for CVaR MDPs using a Bellman operator on L-infinity. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 91248–91275, 2026.

R. Tyrrell Rockafellar and Stanislav Uryasev. Conditional value-at-risk for general loss distributions. *Journal of Banking & Finance*, 26:1443–1471, July 2002. doi: 10.1016/s0378-4266(02)00271-6.

Ilya O. Ryzhov, Martijn R. K. Mes, Warren B. Powell, and Gerald van den Berg. Bayesian exploration for approximate dynamic programming. *Operations Research*, 67(1):198–214, 2019. doi: 10.1287/opre.2018.1772.

David Wingate and Kevin D. Seppi. Prioritization methods for accelerating MDP solvers. *Journal of Machine Learning Research*, 6:851–881, 2005.

A Baseline ADP

Algorithm 1 Baseline ADP

Require: Initial state x_0, threshold η, number of episodes K, number of initialization rollouts N.
Ensure: Cache $\mathcal{C}$.

1: Initialize cache $\mathcal{C} \leftarrow \varnothing$
2: **for** $k = 1, \ldots, K$ **do**
3: $\quad (x, y) \leftarrow (x_0, -\eta)$
4: $\quad$ **for** $t = 0, \ldots, T-1$ **do**
5: $\quad\quad s \leftarrow (t, \mathrm{Round}(x, y))$
6: $\quad\quad$ **for** $a \in \mathcal{A}$ **do**
7: $\quad\quad\quad q(a) \leftarrow \sum_i w_i \textsc{Value}(t+1, f_t(x, a, \xi^{(i)}), y + c_t(x, a))$
8: $\quad\quad$ **end for**
9: $\quad\quad a_t \leftarrow \arg\min_{a \in \mathcal{A}} q(a)$
10: $\quad\quad$ **if** $y < 0$ **then**
11: $\quad\quad\quad \mathcal{C}[s] \leftarrow \min_{a \in \mathcal{A}} q(a)$
12: $\quad\quad$ **end if**
13: $\quad\quad$ Sample ξ_t from Ξ with probabilities w_i
14: $\quad\quad y \leftarrow y + c_t(x, a_t)$
15: $\quad\quad x \leftarrow f_t(x, a_t, \xi_t)$
16: $\quad$ **end for**
17: **end for**
18: **return** $\mathcal{C}$

19: **function** $\textsc{Value}(t, x, y)$
20: $\quad$ **if** $t = T$ **then**
21: $\quad\quad$ **return** $(y + c_T(x))_+$
22: $\quad$ **end if**
23: $\quad$ **if** $y \geq 0$ **then**
24: $\quad\quad$ **return** $y + \hat{V}_t^{\mathrm{RN}}(x)$
25: $\quad$ **end if**
26: $\quad s \leftarrow (t, \mathrm{Round}(x, y))$
27: $\quad$ **if** $s \notin \mathcal{C}$ **then**
28: $\quad\quad \mathcal{C}[s] \leftarrow \textsc{Initialize}(t, x, y, N)$
29: $\quad$ **end if**
30: $\quad$ **return** $\mathcal{C}[s]$
31: **end function**

Here, $\Xi = \{\xi^{(i)}\}_{i=0}^{N_\xi}$ is the discretized noise range, and w_i are the corresponding normalized probability weights $w_i = \frac{p_\xi(\xi^{(i)})}{\sum_j p_\xi(\xi^{(j)})}$.
The resulting policy for a state s can be obtained via lines 6-9 of the algorithm.
The operator $\mathrm{Round}(x, y)$ rounds both state components to six decimal places to construct cache keys, without modifying the actual states. Finally, $\mathrm{Initialize}(t, x, y, N)$ denotes the policy-based value initialization using N Monte Carlo rollouts described in the methodology section.

B Probabilistic ADP

Algorithm 2 Probabilistic ADP

Require: Horizon T, initial state x_0, confidence level α, threshold set $\mathcal{H}$, candidate sets $\mathcal{S}_t$, risk-neutral approximations $\hat{V}_t^{\mathrm{RN}}$, acquisition method A_t, Bellman-backup budgets B_t, initial sample size N_{init}, batch size b

Ensure: Approximate optimal threshold $\hat{\eta}$, objective $\hat{J}(\hat{\eta})$, and policy $\hat{\pi}$

Set $\hat{V}_T(x, y) \leftarrow (y + c_T(x))_+$
if A_t uses reachability or tail relevance **then**
    Generate rollouts under the risk-neutral policy using a reference threshold η_{ref}
    Compute rollout-based acquisition scores
end if
for $t = T - 1, \ldots, 1$ **do**
    Set $n_t \leftarrow \min\{|\mathcal{S}_t|, \lfloor B_t/|\mathcal{A}| \rfloor\}$
    if A_t is random sampling **then**
        Select n_t states randomly from $\mathcal{S}_t$
    else
        Select $\min(N_{\mathrm{init}}, n_t)$ states randomly from $\mathcal{S}_t$
    end if
    Compute Bellman targets for all actions using (21)
    Fit one GP per action using the selected states
    while the number of selected states is less than n_t **do**
        Compute $A_t(s)$ for all remaining candidates
        Select up to b states with the highest scores
        Compute their Bellman targets for all actions
        Add the new observations to the training data
        Refit the GP models
    end while
    Construct $\hat{V}_t$ using (24) for $y < 0$ and risk-neutral continuation for $y \geq 0$
end for
for $\eta \in \mathcal{H}$ **do**
    Compute $\hat{Q}_0(x_0, -\eta, a)$ for all $a \in \mathcal{A}$ using the Bellman operator
    $\hat{V}_0(x_0, -\eta) \leftarrow \min_{a \in \mathcal{A}} \hat{Q}_0(x_0, -\eta, a)$
    $\hat{J}(\eta) \leftarrow \eta + \frac{1}{1-\alpha} \hat{V}_0(x_0, -\eta)$
end for
$\hat{\eta} \leftarrow \arg\min_{\eta \in \mathcal{H}} \hat{J}(\eta)$
Construct $\hat{\pi}$ using the minimizing root action, GP policies for $y < 0$, and risk-neutral policies for $y \geq 0$
return $\hat{\eta}, \hat{J}(\hat{\eta}), \hat{\pi}$